

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ - THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP
BÃI ĐỖ XE THÔNG MINH ỨNG DỤNG CÔNG
NGHỆ XỬ LÝ ẢNH NHẬN DẠNG BIỂN SỐ XE

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

GVHD : TS.Huỳnh Thế Thiện

SVTH : Nguyễn Hải Đăng

MSSV : 22139011

Khóa : 2022

TP. HỒ CHÍ MINH – 6/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ - THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP
BÃI ĐỖ XE THÔNG MINH ỨNG DỤNG CÔNG
NGHỆ XỬ LÝ ẢNH NHẬN DẠNG BIỂN SỐ XE

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

GVHD : TS.Huỳnh Thế Thiện

SVTH : Nguyễn Hải Đăng

MSSV : 22139011

Khóa : 2022

TP. HỒ CHÍ MINH – 6/2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: BÀI ĐỒ XE THÔNG MINH ỨNG DỤNG CÔNG NGHỆ XỬ LÝ ẢNH NHẬN DẠNG BIỂN

SỐ XE

Sinh viên: Nguyễn Hải Đăng

MSSV: 22139011

Hướng dẫn: TS. Huỳnh Thế Thiện

Nhận xét bao gồm các nội dung sau đây:

- 1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:** Đề tài giải quyết được bài toán quản lý bãi đỗ xe thông minh ở mức độ mô phỏng cơ bản. Hướng tiếp cận dùng xử lý ảnh cổ điển và OCR có ưu điểm là chi phí thấp, tuy nhiên chưa mang nhiều ý nghĩa đột phá về mặt khoa học do các công cụ này đã khá phổ biến và còn hạn chế trong điều kiện thực tế.
- 2. Phương pháp thực hiện/ phân tích/ thiết kế:** Sinh viên hiểu và vận dụng được kiến trúc hệ thống cơ bản, kết hợp phần cứng Arduino và phần mềm trên máy tính. Tuy nhiên, các giải pháp kỹ thuật dừng ở mức cơ bản, thiết kế mô hình dự đoán số chỗ trống (Seasonal-Mean) còn khá chung chung và chưa khai thác sâu yếu tố dữ liệu.
- 3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:** Mô hình phần cứng đã vận hành được các chức năng cốt lõi (kiểm soát barrier, quét thẻ, giao diện dashboard). Tuy nhiên, kết quả nhận diện biển số chưa cao (chỉ đạt 60% trong điều kiện thử nghiệm), dễ bị sai lệch do môi trường và góc chụp. Hệ thống phù hợp để minh chứng ý tưởng (proof-of-concept) hơn là ứng dụng thực tế.
- 4. Kết luận và đề xuất:** Phần kết luận bám sát các số liệu thu được. Sinh viên có đề xuất các hướng mở rộng (IoT, học sâu), nhưng đây mới chỉ là định hướng chung chung, chưa có phân tích sâu về tính khả thi hay kiến trúc triển khai cụ thể.
- 5. Hình thức trình bày và bố cục báo cáo:** Cấu trúc báo cáo khá đầy đủ các chương mục. Dù vậy, trong quá trình thực hiện sinh viên còn lỗi trình bày và đánh máy, và lỗi định dạng mục lục. Sau khi được nhắc nhở, sinh viên đã khắc phục được phần lớn các lỗi này ở bản nộp cuối.
- 6. Kỹ năng chuyên nghiệp và tính sáng tạo:** Sinh viên có nỗ lực tự tìm hiểu kỹ năng lập trình (Python, C++) và lắp ráp phần cứng. Kỹ năng xử lý thuật toán (chống lặp sự kiện) khá tốt. Sinh viên cần cẩn trọng trong nghiên cứu khoa học và trình bày văn bản học thuật cần phải rèn luyện thêm nhiều.
- 7. Tài liệu trích dẫn:** Có sử dụng tài liệu tham khảo phù hợp với nội dung nghiên cứu. Việc trích dẫn tài liệu ở bản thảo cuối đã tuân thủ tương đối định dạng quy định, dù ban đầu còn nhiều thiếu sót.
- 8. Đánh giá về sự trung lập của đề tài:** Đề tài là nỗ lực tự tích hợp và thực hiện của sinh viên, không có dấu hiệu sao chép toàn phần. Việc ứng dụng các thư viện mã nguồn mở có sẵn được ghi nhận rõ ràng.
- 9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa:** Hệ thống còn bộc lộ điểm yếu lớn ở khâu nhận diện và thiếu tính ổn định. Sinh viên đã tiếp thu và sửa chữa các sai sót cực kỳ nghiêm trọng về công thức toán học (MAE, RMSE, Otsu) theo yêu cầu của GVHD để đảm bảo tính chính xác tối thiểu cho báo cáo.
- 10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên:** Sinh viên có thái độ cầu thị, biết lắng nghe và nhanh chóng chỉnh sửa các lỗi sai khi được giảng viên hướng dẫn chỉ ra. Hoàn thành khối lượng công việc đúng thời hạn.

Đề nghị của giảng viên hướng dẫn:

Báo cáo hoàn toàn đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp trước Hội đồng.

Tp. HCM, ngày 23 tháng 06 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)



TS. Huỳnh Thế Thiện

LỜI CAM ĐOAN

Em tên là: **Nguyễn Hải Đăng** – MSSV: **22139011**

Em xin cam đoan đề tài đồ án tốt nghiệp “**BÃI ĐỖ XE THÔNG MINH ỨNG DỤNG CÔNG NGHỆ XỬ LÝ ẢNH NHẬN DẠNG BIỂN SỐ XE**” là công trình do em trực tiếp tìm hiểu, thiết kế, thi công phần cứng - phần mềm và viết báo cáo, dưới sự hướng dẫn của thầy TS. Huỳnh Thế Thiện.

Các số liệu, kết quả trình bày trong báo cáo được em thu thập và đo đạc trực tiếp từ hệ thống thực tế do em xây dựng, không sao chép từ bất kỳ công trình nào khác. Những nội dung, số liệu tham khảo từ tài liệu, bài báo bên ngoài đều được trích dẫn nguồn gốc đầy đủ và liệt kê trong phần Tài liệu tham khảo của từng chương.

Em xin chịu hoàn toàn trách nhiệm về nội dung của báo cáo này. Nếu phát hiện có bất kỳ sự sao chép, gian lận số liệu hay hành vi không trung thực nào, em xin chấp nhận mọi hình thức kỷ luật theo quy định của Nhà trường.

TP. Hồ Chí Minh, ngày ... tháng ... năm 2026

Người thực hiện đồ án

Nguyễn Hải Đăng

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện đồ án tốt nghiệp, em đã nhận được rất nhiều sự quan tâm và giúp đỡ từ quý thầy cô cùng những người xung quanh. Em xin gửi lời cảm ơn chân thành đến quý thầy cô Khoa Điện – Điện tử, Trường Đại học Sư phạm Kỹ thuật Thành phố Hồ Chí Minh đã tận tình giảng dạy, truyền đạt những kiến thức và kỹ năng nền tảng trong suốt thời gian em học tập tại trường. Đây là hành trang quý báu giúp em có đủ năng lực để thực hiện đề tài này.

Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến thầy TS. Huỳnh Thế Thiện. Người đã trực tiếp hướng dẫn, định hướng đề tài và luôn theo sát, góp ý kịp thời mỗi khi em gặp khó khăn trong quá trình thiết kế, thi công và viết báo cáo. Sự tận tâm, nhiệt huyết của thầy là nguồn động lực rất lớn giúp em hoàn thành đồ án này.

Em cũng xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ em trong suốt thời gian thực hiện đồ án.

Do kiến thức và kinh nghiệm thực tế còn hạn chế, đồ án khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của quý thầy cô để đề tài được hoàn thiện hơn. Em xin trân trọng cảm ơn!

TÓM TẮT

Bãi đỗ xe truyền thống vận hành bằng vé giấy và nhân viên trông giữ thủ công, dễ sai sót, thất lạc vé, mất cắp phương tiện và gây ùn tắc tại cổng ra/vào khi lưu lượng xe tăng cao; người quản lý cũng thường thiếu công cụ theo dõi tình trạng lấp đầy theo thời gian thực và dự báo nhu cầu chỗ đỗ trong các khung giờ sắp tới.

Xuất phát từ thực tế đó, đề án xây dựng một hệ thống quản lý bãi giữ xe tự động hóa khâu nhận diện phương tiện, kiểm soát ra/vào và giám sát bãi xe theo thời gian thực, theo kiến trúc phân lớp: Arduino đảm nhiệm cảm biến và điều khiển cơ cấu chấp hành tại cổng, máy tính vừa xử lý ảnh vừa là máy chủ cục bộ (lưu trữ, giám sát web), giao tiếp với Arduino qua serial (USB) phù hợp với mô hình tập trung tại cổng, nơi camera, vi điều khiển và máy chủ đặt gần nhau. Các chức năng chính của hệ thống bao gồm:

1. Định danh phương tiện bằng thẻ RFID tại hai cổng vào và ra.
2. Nhận diện biển số tự động bằng OpenCV (định vị/cắt vùng biển số) và Tesseract OCR (đọc ký tự trong vùng đã cắt).
3. Điều khiển barrier tự động bằng động cơ servo khi xe vào/ra hợp lệ.
4. Ánh xạ (mapping) thẻ RFID với biển số nhận diện lúc xe vào, đối chiếu lúc xe ra để chống gian lận và mất xe.
5. Hiển thị trạng thái từng ô đỗ bằng đèn LED trên mô hình và cập nhật trên giao diện giám sát.
6. Giám sát, lưu trữ qua web: máy chủ Flask cung cấp API đọc/ghi SQLite, phục vụ dashboard hiển thị bản đồ ô đỗ, số chỗ trống, trạng thái cổng và lịch sử ra/vào theo thời gian thực.
7. Phân tích dữ liệu lịch sử, dự đoán số chỗ trống theo khung giờ bằng mô hình nền seasonal-mean (trung bình theo giờ và thứ trong tuần), đánh giá bằng MAE/RMSE trên dữ liệu chia theo thời gian.

Hệ thống đã thi công và vận hành hoàn chỉnh ở quy mô mô hình ba ô đỗ, với vòng phát hiện xe - nhận diện biển số - đối chiếu RFID - điều khiển barrier - lưu trữ - giám sát web - dự đoán chỗ trống hoạt động thông suốt, đạt kết quả đúng ở điều kiện thuận lợi (còn sai số khi ảnh mờ/thiếu sáng, đặc điểm của xử lý ảnh cổ điển chưa dùng học sâu), độ trễ giám sát ở mức giây, và sai số dự đoán seasonal-mean phù hợp quy mô dữ liệu hiện có; các hạn chế được trình bày trung thực làm cơ sở cho hướng phát triển.

MỤC LỤC

LỜI CAM ĐOAN	ii
LỜI CẢM ƠN	iii
TÓM TẮT	iv
MỤC LỤC.....	vi
DANH MỤC HÌNH ẢNH	x
DANH MỤC BẢNG.....	xii
DANH MỤC TỪ VIẾT TẮT.....	xiii
CHƯƠNG 1	1
TỔNG QUAN	1
1.1 GIỚI THIỆU	1
1.2. MỤC TIÊU ĐỀ TÀI.....	2
1.3. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU	3
1.3.1. Đối tượng nghiên cứu	3
1.3.2. Phạm vi nghiên cứu.....	3
1.4. PHƯƠNG PHÁP NGHIÊN CỨU	4
1.4.1. Nghiên cứu và thu thập tài liệu	4
1.4.2. Thiết kế, triển khai và đánh giá hệ thống	5
1.5. GIỚI HẠN ĐỀ TÀI	5
1.6. BỐ CỤC CỦA BÁO CÁO.....	6
CHƯƠNG 2	8
CƠ SỞ LÝ THUYẾT	8
2.1. THỊ GIÁC MÁY TÍNH VÀ XỬ LÝ ẢNH TRONG NHẬN DIỆN BIỂU SỐ	8

2.1.1. Ảnh số, không gian màu và chuyển ảnh xám	8
2.1.2. Tiền xử lý và nhị phân hóa ảnh	9
2.1.3. Phát hiện biên và tìm đường viền	10
2.1.4. Vai trò của OpenCV trong hệ thống	11
2.2. NHẬN DẠNG KÝ TỰ QUANG HỌC VỚI TESSERACT OCR	11
2.2.1. Khái niệm OCR.....	11
2.2.2. Tesseract OCR	11
2.2.3. Đặc thù OCR trong nhận diện biên số	12
2.3. VI ĐIỀU KHIỂN ARDUINO VÀ GIAO TIẾP SERIAL.....	13
2.3.1. Vi điều khiển Arduino trong hệ thống nhúng	13
2.3.2. Tín hiệu vào/ra số và thiết bị ngoại vi.....	13
2.3.3. Giao tiếp Serial UART.....	14
2.4. KIẾN TRÚC MÁY CHỦ CỤC BỘ, FLASK VÀ SQLITE.....	15
2.4.1. Mô hình client-server cục bộ	15
2.4.2. Flask và REST API.....	15
2.4.3. SQLite và mô hình dữ liệu quan hệ	16
2.4.4. Đa luồng và cơ chế cập nhật dữ liệu bằng polling.....	16
2.5. MÔ HÌNH DỰ ĐOÁN SỐ CHỖ TRỐNG SEASONAL-MEAN	17
2.5.1. Chuỗi thời gian và tính mùa vụ.....	17
2.5.2. Mô hình Seasonal-Mean	17
2.5.3. Đánh giá mô hình dự đoán.....	18
2.5.4. Chia dữ liệu huấn luyện và kiểm thử theo thời gian	19
CHƯƠNG 3	20
THIẾT KẾ HỆ THỐNG.....	20
3.1. YÊU CẦU VÀ SƠ ĐỒ KHỐI HỆ THỐNG	20

3.1.1. Yêu cầu hệ thống.....	20
3.1.2. Sơ đồ khối và chức năng của từng khối	21
3.1.3. Mô tả hoạt động của hệ thống	24
3.2. THIẾT KẾ PHẦN CỨNG	27
3.2.1. Khối xử lý trung tâm	27
3.2.2. Khối điều khiển thiết bị ngoại vi.....	28
3.2.3. Khối đọc thẻ RFID	31
3.2.4. Khối động cơ.....	34
3.2.5. Khối đèn báo và còi cảnh báo	35
3.2.6. Khối camera	37
3.2.7. Khối cảm biến hồng ngoại	38
3.2.8. Khối nguồn.....	40
3.3. THIẾT KẾ PHẦN MỀM	42
3.3.1. Lưu đồ chương trình Arduino	43
3.3.2. Lưu đồ chương trình ứng dụng trên máy tính.....	45
3.3.2.1. Luồng cập nhật trạng thái ô đỗ	47
3.3.2.2. Luồng ghi sự kiện xe vào	47
3.3.2.3. Luồng ghi sự kiện xe ra.....	47
3.3.3. Module nhận diện biển số xe	48
3.3.4. Thiết kế cơ sở dữ liệu.....	51
3.3.5. Thiết kế web dashboard	52
3.3.6. Mô hình dự đoán chỗ trống.....	56
3.3.6.1. Xây dựng bảng tra cứu từ dữ liệu lịch sử.....	56
3.3.6.2. Dự đoán cho 24 giờ tiếp theo	56
3.3.6.3. Đánh giá độ chính xác của mô hình.....	57
CHƯƠNG 4	58

KẾT QUẢ	58
4.1. KẾT QUẢ THỰC HIỆN MÔ HÌNH	58
4.2. KẾT QUẢ NHẬN DIỆN BIÊN SỐ.....	62
4.3. KẾT QUẢ DASHBOARD GIÁM SÁT 3 Ô ĐỒ	65
4.4. KẾT QUẢ MÔ HÌNH DỰ ĐOÁN CHỖ TRỐNG	68
4.5. KẾT LUẬN.....	69
CHƯƠNG 5	71
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	71
5.1. KẾT LUẬN.....	71
5.2. HƯỚNG PHÁT TRIỂN.....	73
TÀI LIỆU THAM KHẢO.....	76

DANH MỤC HÌNH ẢNH

Hình 3.1. Sơ đồ khối tổng thể của hệ thống.....	22
Hình 3.2. Arduino Nano.....	30
Hình 3.3. Ký hiệu Arduino Nano.....	30
Hình 3.4. Module RFID RC522.....	32
Hình 3.5. Ký hiệu module RFID RC522	33
Hình 3.6. Động cơ micro servo SG90.....	34
Hình 3.7. Ký hiệu module servo SG90	34
Hình 3.8. Cảm biến hồng ngoại IR01	39
Hình 3.9. Ký hiệu cảm biến hồng ngoại	39
Hình 3.10. Lưu đồ chương trình Arduino	44
Hình 3.11. Lưu đồ vòng lặp xử lý chính trên máy tính	46
Hình 3.12. Lưu đồ module nhận diện biển số xe	51
Hình 3.13. Giao diện web dashboard	55
Hình 4.1. Mica.....	58
Hình 4.2. Lắp đặt cổng vào ra.....	58
Hình 4.3. Lắp đặt bãi đỗ.....	59
Hình 4.4. Gia công mạch điều khiển.....	59
Hình 4.5. Lắp đặt các cảm biến, module đọc thẻ từ.....	60
Hình 4.6. Lắp đặt đèn báo và camera.....	60
Hình 4.7. Lắp đặt mạch điện	61
Hình 4.8. Mô hình thử nghiệm sau khi hoàn thiện	62

Hình 4.9. Ví dụ một lượt nhận diện đúng ảnh chụp biển số, ảnh sau khi OpenCV định vị/cắt khung biển, và chuỗi ký tự Tesseract OCR đọc được	64
Hình 4.10. Ví dụ một lượt nhận diện sai do ảnh mờ/thiếu sáng đối chiếu biển số thực tế và kết quả OCR.....	65
Hình 4.11. Giao diện dashboard bản đồ 3 ô đỗ, số chỗ trống, trạng thái barrier theo thời gian thực	67
Hình 4.12. Biểu đồ thống kê lượt xe theo ngày, theo giờ cao điểm và doanh thu trên dashboard	68
Hình 4.13. Biểu đồ dự đoán số chỗ trống 24 giờ tới trên dashboard	69

DANH MỤC BẢNG

Bảng 3.1. Thông số kỹ thuật của Arduino Nano.....	31
Bảng 3.2. Thông số kỹ thuật của RFID RC522	33
Bảng 3.3. Thông số kỹ thuật của servo SG90.....	35
Bảng 3.4. Thông số kỹ thuật của Webcam HD 720.....	38
Bảng 3.5. Thông số kỹ thuật của cảm biến hồng ngoại	40
Bảng 3.6. Điện áp và dòng tiêu thụ ước tính của các thành phần trong hệ thống.....	41
Bảng 3.7. Lược đồ ba bảng dữ liệu trong parking.db	51
Bảng 3.8. Các API do máy chủ Flask cung cấp.....	53
Bảng 3.9. Tần suất truy vấn (polling) của giao diện web	53
Bảng 3.10. Cấu trúc 7 phần của trang web dashboard.....	54
Bảng 3.11. Ví dụ một số dòng trong bảng tra cứu trung bình theo (thứ, giờ)	56
Bảng 4.1. Kết quả thử nghiệm nhận diện biển số trên các điều kiện chụp khác nhau.....	63
Bảng 4.2. Đối chiếu trạng thái thực tế của 3 ô đỗ với trạng thái hiển thị trên dashboard	66
Bảng 4.3. Kết quả đánh giá mô hình dự đoán seasonal-mean trên dữ liệu thử nghiệm.....	69

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Từ đầy đủ
ALPR/ANPR	Automatic License Plate Recognition
API	Application Programming Interface
ARIMA	AutoRegressive Integrated Moving Average
BGR	Blue Green Red
CDN	Content Delivery Network
CRNN	Convolutional Recurrent Neural Network
CSS	Cascading Style Sheets
DC	Direct Current
EEPROM	Electrically Erasable Programmable Read-Only Memory
GPIO	General Purpose Input/Output
HTML	HyperText Markup Language
IC	Integrated Circuit
IoT	Internet of Things
JSON	JavaScript Object Notation
LED	Light Emitting Diode
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MQTT	Message Queuing Telemetry Transport
OCR	Optical Character Recognition
OEM	OCR Engine Mode
PCB	Printed Circuit Board
PSM	Page Segmentation Mode
PWM	Pulse Width Modulation

QR	Quick Response
REST	Representational State Transfer
RFID	Radio Frequency Identification
RGB	Red Green Blue
RMSE	Root Mean Squared Error
SPI	Serial Peripheral Interface
SQL	Structured Query Language
SRAM	Static Random Access Memory
TTL	Transistor-Transistor Logic
UART	Universal Asynchronous Receiver-Transmitter
UID	Unique Identifier
UPS	Uninterruptible Power Supply
URL	Uniform Resource Locator
USB	Universal Serial Bus
YOLO	You Only Look Once

CHƯƠNG 1

TỔNG QUAN

1.1 GIỚI THIỆU

Trong những năm gần đây, cùng với sự phát triển của khoa học kỹ thuật và nhu cầu đi lại ngày càng tăng, số lượng phương tiện giao thông tại các khu vực đô thị, trường học, cơ quan, trung tâm thương mại và bãi giữ xe ngày càng lớn. Điều này đặt ra yêu cầu cao hơn đối với công tác quản lý phương tiện ra vào, kiểm soát thời gian gửi xe, giám sát số lượng chỗ trống và hạn chế các rủi ro như thất lạc thẻ, nhầm lẫn phương tiện hoặc mất cắp xe.

Phương pháp quản lý bãi giữ xe truyền thống thường phụ thuộc nhiều vào con người, chẳng hạn như ghi vé thủ công, kiểm tra biển số bằng mắt thường hoặc thống kê số lượng xe theo kinh nghiệm. Cách làm này có thể phù hợp với quy mô nhỏ, tuy nhiên khi số lượng phương tiện tăng lên, quá trình vận hành dễ phát sinh sai sót, mất thời gian và khó lưu trữ dữ liệu phục vụ thống kê, quản lý lâu dài. Vì vậy, việc ứng dụng các hệ thống tự động vào quản lý bãi giữ xe là hướng tiếp cận cần thiết và có ý nghĩa thực tiễn.

Trong các hệ thống quản lý bãi giữ xe thông minh, nhận dạng biển số xe là một trong những chức năng quan trọng. Thông qua hình ảnh thu được từ camera, hệ thống có thể xử lý ảnh, xác định vùng biển số và nhận dạng chuỗi ký tự trên biển số. Khi kết hợp với các công nghệ khác như thẻ RFID, cảm biến hồng ngoại, cơ sở dữ liệu và giao diện giám sát, hệ thống có thể hỗ trợ kiểm soát xe ra/vào, lưu trữ lịch sử vận hành, tính phí gửi xe và theo dõi trạng thái bãi xe theo thời gian thực.

Từ những vấn đề trên, đề tài “Bãi đỗ xe thông minh ứng dụng công nghệ xử lý ảnh nhận dạng biển số xe” được thực hiện nhằm xây dựng một mô hình hệ thống quản lý bãi giữ xe tự động ở quy mô nhỏ. Hệ thống tập trung vào việc kết hợp giữa xử lý ảnh nhận dạng biển số, xác thực bằng thẻ RFID, giám sát trạng thái

ô đồ và lưu trữ dữ liệu vận hành, qua đó minh chứng tính khả thi của việc ứng dụng công nghệ vào quản lý bãi giữ xe.

1.2. MỤC TIÊU ĐỀ TÀI

Mục tiêu tổng thể của đề tài là xây dựng một mô hình hệ thống quản lý bãi giữ xe thông minh có khả năng kiểm soát phương tiện ra/vào, nhận diện biển số xe, giám sát trạng thái bãi xe, lưu trữ dữ liệu vận hành và hỗ trợ người quản lý theo dõi hoạt động của bãi xe thông qua giao diện web.

Cụ thể, đề tài hướng đến các mục tiêu sau:

- Xây dựng chức năng nhận diện biển số xe tự động từ hình ảnh camera bằng thư viện OpenCV kết hợp công cụ Tesseract OCR, theo hướng xử lý ảnh cổ điển, không sử dụng mô hình học sâu.
- Kết hợp thẻ RFID với biển số đã nhận diện để quản lý xe ra/vào, hỗ trợ đổi chiều thông tin phương tiện khi xe ra khỏi bãi và giảm rủi ro nhầm lẫn hoặc gian lận.
- Thiết kế hệ thống điều khiển cổng vào/ra bằng vi điều khiển Arduino, đầu đọc RFID, cảm biến hồng ngoại và servo điều khiển barrier.
- Xây dựng giao diện web dashboard để giám sát trạng thái bãi xe theo thời gian thực, bao gồm trạng thái từng ô đỗ, số chỗ trống, trạng thái cổng vào/ra và lịch sử xe ra/vào gần nhất.
- Lưu trữ dữ liệu vận hành của hệ thống như biển số xe, mã thẻ RFID, thời gian vào/ra, phí gửi xe và trạng thái ô đỗ vào cơ sở dữ liệu cục bộ.
- Xây dựng chức năng phân tích dữ liệu lịch sử và dự đoán số chỗ trống ở mức cơ bản dựa trên phương pháp trung bình mùa vụ, nhằm hỗ trợ người quản lý quan sát xu hướng lưu lượng xe theo từng khung giờ.

1.3. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

1.3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là mô hình hệ thống quản lý bãi giữ xe thông minh, tập trung vào quá trình nhận diện phương tiện, kiểm soát xe ra/vào, giám sát trạng thái ô đỗ và lưu trữ dữ liệu vận hành.

Các thành phần và công nghệ chính được nghiên cứu, triển khai trong đề tài gồm:

- Các kỹ thuật xử lý ảnh cổ điển trong OpenCV như chuyển ảnh xám, nhị phân hóa, lọc nhiễu, tìm đường viền và xác định vùng biển số.
- Công cụ nhận dạng ký tự quang học Tesseract OCR dùng để đọc chuỗi ký tự biển số từ vùng ảnh đã được xử lý.
- Vi điều khiển Arduino trong việc đọc dữ liệu từ cảm biến hồng ngoại, đầu đọc RFID và điều khiển servo barrier.
- Cơ chế giao tiếp serial giữa Arduino và máy tính để trao đổi dữ liệu vận hành.
- Cơ sở dữ liệu SQLite dùng để lưu trữ thông tin phương tiện, lịch sử ra/vào và trạng thái bãi xe.
- Máy chủ web cục bộ sử dụng Flask để xây dựng giao diện giám sát và quản lý hệ thống.
- Phương pháp dự đoán số chỗ trống dựa trên trung bình mùa vụ từ dữ liệu lịch sử ra/vào tích lũy được.

1.3.2. Phạm vi nghiên cứu

Đề tài được xây dựng và thử nghiệm trên mô hình mô phỏng bãi giữ xe quy mô nhỏ, gồm ba ô đỗ được giám sát bằng cảm biến hồng ngoại và hai cổng vào/ra riêng biệt. Mỗi cổng sử dụng một đầu đọc RFID và một servo để điều khiển barrier.

Trong quá trình thử nghiệm, xe mô hình được sử dụng để kiểm tra hoạt động vận hành của hệ thống. Riêng khối nhận diện biển số được kiểm tra thêm trên

tập ảnh biển số tĩnh nhằm đánh giá độc lập khả năng nhận diện của phần mềm xử lý ảnh, không phụ thuộc hoàn toàn vào quá trình vận hành toàn hệ thống.

Biển số được nhận diện trong phạm vi đề tài là biển số mô phỏng theo dạng hai hàng, chữ đen trên nền trắng, gồm tám ký tự, không bị che khuất, tróc sơn hoặc rỉ sét nghiêm trọng. Ảnh đầu vào cần có độ nét tương đối, ánh sáng phù hợp và góc chụp nằm trong giới hạn cho phép. Đây là các điều kiện phù hợp với một pipeline xử lý ảnh cổ điển kết hợp OCR. Trong các trường hợp ảnh quá mờ, quá chói, biển số bị nghiêng nhiều hoặc ký tự bị che khuất, độ chính xác nhận diện có thể giảm.

Đề tài không mở rộng sang quy mô bãi xe lớn nhiều tầng hoặc nhiều khu vực, không triển khai hệ thống thanh toán điện tử thực tế và không xây dựng ứng dụng di động cho người dùng tìm kiếm hoặc đặt chỗ từ xa. Các nội dung này được xem là hướng phát triển tiếp theo của hệ thống.

1.4. PHƯƠNG PHÁP NGHIÊN CỨU

1.4.1. Nghiên cứu và thu thập tài liệu

Đề tài tiến hành thu thập, nghiên cứu các tài liệu lý thuyết và tài liệu kỹ thuật liên quan đến những công nghệ được sử dụng trong hệ thống. Nội dung nghiên cứu bao gồm xử lý ảnh số và thị giác máy tính cổ điển với OpenCV, nguyên lý hoạt động của công cụ Tesseract OCR, giao tiếp serial, lập trình vi điều khiển Arduino, cơ sở dữ liệu SQLite, máy chủ web Flask và các phương pháp dự báo chuỗi thời gian cơ bản.

Nguồn tài liệu được tham khảo gồm giáo trình, bài báo khoa học, tài liệu kỹ thuật và tài liệu chính thức của các công nghệ liên quan. Các nội dung lý thuyết này được trình bày chi tiết hơn ở Chương 2 của báo cáo.

1.4.2. Thiết kế, triển khai và đánh giá hệ thống

Quá trình thực hiện đề tài được chia thành ba giai đoạn chính: thiết kế hệ thống, triển khai hệ thống và đánh giá kết quả.

Ở giai đoạn thiết kế, đề tài xác định yêu cầu chức năng và phi chức năng của hệ thống, xây dựng sơ đồ khối tổng thể, phân chia nhiệm vụ giữa phần cứng và phần mềm. Trong đó, Arduino đảm nhiệm việc đọc cảm biến, đọc thẻ RFID và điều khiển barrier; máy tính đảm nhiệm việc xử lý ảnh nhận diện biển số, lưu trữ dữ liệu, vận hành máy chủ web và hiển thị giao diện giám sát.

Ở giai đoạn triển khai, đề tài tiến hành lập trình firmware cho Arduino, xây dựng pipeline nhận diện biển số bằng OpenCV kết hợp Tesseract OCR, thiết kế cơ sở dữ liệu SQLite, xây dựng máy chủ Flask và giao diện web dashboard. Đồng thời, chức năng dự đoán số chỗ trống được xây dựng dựa trên dữ liệu lịch sử của hệ thống.

Ở giai đoạn đánh giá, đề tài tiến hành kiểm tra từng khối chức năng riêng lẻ và đánh giá hoạt động của toàn hệ thống. Khối nhận diện biển số được thử nghiệm trên tập ảnh tĩnh; khối điều khiển cổng được kiểm tra với RFID, cảm biến và servo; hệ thống hoàn chỉnh được vận hành trên mô hình bãi giữ xe ba ô đỗ. Kết quả thử nghiệm, các hạn chế quan sát được và hướng khắc phục được trình bày ở Chương 4.

1.5. GIỚI HẠN ĐỀ TÀI

Do giới hạn về thời gian, điều kiện thiết bị và phạm vi thực hiện đồ án, đề tài tập trung xây dựng mô hình thử nghiệm nhằm minh chứng tính khả thi của hệ thống. Một số giới hạn của đề tài gồm:

- Quy mô mô hình còn nhỏ: hệ thống được triển khai và thử nghiệm trên mô hình mô phỏng gồm ba ô đỗ, chưa thử nghiệm trong môi trường bãi xe thực tế với số lượng phương tiện lớn và điều kiện vận hành phức tạp.
- Kiến trúc truyền dữ liệu còn đơn giản: Arduino và máy tính giao tiếp với nhau thông qua kết nối serial USB. Cách triển khai này phù hợp với mô

hình tập trung, trong đó camera, vi điều khiển và máy chủ đặt gần nhau. Tuy nhiên, khi mở rộng sang bãi xe lớn có nhiều khu vực hoặc cảm biến phân tán, hệ thống cần được nâng cấp lớp truyền thông.

- Độ chính xác nhận diện biển số phụ thuộc vào chất lượng ảnh đầu vào: phương pháp xử lý ảnh cổ điển kết hợp Tesseract OCR nhạy với điều kiện ánh sáng, độ nét, góc chụp và tình trạng vật lý của biển số. Khi ảnh đầu vào không đạt yêu cầu, kết quả định vị vùng biển số hoặc nhận dạng ký tự có thể bị sai lệch.
- Máy chủ sử dụng trong đề tài là máy tính cá nhân: Flask và SQLite được triển khai trên máy tính dùng cho phát triển và thử nghiệm, chưa phải là máy chủ công nghiệp hoặc hạ tầng cloud như trong các hệ thống thương mại.
- Chức năng dự đoán mới ở mức cơ bản: mô hình dự đoán số chỗ trống sử dụng phương pháp trung bình mùa vụ, được xây dựng trên lượng dữ liệu lịch sử còn hạn chế. Do đó, chức năng này chủ yếu nhằm minh họa khả năng khai thác dữ liệu vận hành, chưa thể hiện đầy đủ năng lực dự báo trong điều kiện thực tế phức tạp.
- Chưa triển khai các chức năng mở rộng thương mại: đề tài chưa xây dựng hệ thống thanh toán điện tử thực tế, chưa có ứng dụng di động cho người dùng xem chỗ trống hoặc đặt chỗ từ xa. Các chức năng này được xem là hướng phát triển trong tương lai.

1.6. BỐ CỤC CỦA BÁO CÁO

Nội dung báo cáo được trình bày trong năm chương như sau:

Chương 1 - Tổng quan: Trình bày bối cảnh hình thành đề tài, mục tiêu nghiên cứu, đối tượng và phạm vi nghiên cứu, phương pháp thực hiện, giới hạn đề tài và bố cục của báo cáo.

Chương 2 - Cơ sở lý thuyết: Trình bày các nền tảng lý thuyết và công nghệ được sử dụng trong hệ thống, bao gồm xử lý ảnh bằng OpenCV, nhận dạng ký tự quang học bằng Tesseract OCR, vi điều khiển Arduino, giao tiếp serial, RFID, cảm biến hồng ngoại, servo, Flask, SQLite và phương pháp dự đoán trung bình mùa vụ.

Chương 3 - Thiết kế hệ thống: Trình bày yêu cầu hệ thống, sơ đồ khối tổng thể, chức năng của từng khối, thiết kế phần cứng, thiết kế phần mềm, cơ sở dữ liệu, giao diện web dashboard và lưu đồ xử lý của hệ thống.

Chương 4 - Kết quả: Trình bày kết quả thử nghiệm nhận diện biển số, kết quả vận hành công vào/ra, kết quả giám sát trạng thái bãi xe, kết quả lưu trữ dữ liệu, kết quả chức năng dự đoán số chỗ trống và đánh giá các hạn chế trong quá trình thử nghiệm.

Chương 5 - Kết luận và hướng phát triển: Tổng kết những kết quả đã đạt được so với mục tiêu ban đầu, nêu các hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo cho hệ thống.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1. THỊ GIÁC MÁY TÍNH VÀ XỬ LÝ ẢNH TRONG NHẬN DIỆN BIỂN SỐ

Nhận diện biển số tự động (Automatic License Plate Recognition - ALPR/ANPR) là bài toán kết hợp giữa thị giác máy tính và nhận dạng ký tự. Về tổng thể, một hệ thống ALPR thường gồm ba giai đoạn chính: thu nhận ảnh phương tiện, xác định vùng chứa biển số và đọc chuỗi ký tự trên biển số. Các nghiên cứu về ALPR thường chia phương pháp nhận diện thành hai nhóm: nhóm sử dụng đặc trưng ảnh cổ điển như biên cạnh, hình thái học, màu sắc và hình dạng; và nhóm sử dụng mô hình học sâu để định vị hoặc nhận dạng trực tiếp từ ảnh [1].

Trong phạm vi đồ án, hệ thống sử dụng hướng tiếp cận cổ điển: thư viện OpenCV được dùng để tiền xử lý ảnh và xác định vùng biển số, sau đó vùng ảnh biển số được chuyển sang Tesseract OCR để nhận dạng ký tự. Do đó, các khái niệm quan trọng cần trình bày gồm ảnh số, chuyển ảnh xám, nhị phân hóa, phát hiện đường viền và các đặc trưng hình học dùng để chọn vùng biển số.

2.1.1. Ảnh số, không gian màu và chuyển ảnh xám

Ảnh số là ma trận hai chiều gồm nhiều điểm ảnh (pixel). Với ảnh màu thông thường, mỗi điểm ảnh thường được biểu diễn bởi ba thành phần màu. Trong mô hình RGB, ba kênh màu tương ứng với đỏ (R), xanh lá (G) và xanh dương (B). Trong OpenCV, ảnh màu thường được lưu theo thứ tự kênh BGR, tuy nhiên về bản chất vẫn là biểu diễn ba thành phần màu của mỗi điểm ảnh.

Đối với bài toán nhận diện biển số bằng đặc trưng hình học, thông tin quan trọng nhất thường là sự khác biệt sáng - tối giữa ký tự, nền biển số và môi trường xung quanh. Vì vậy, ảnh màu thường được chuyển sang ảnh xám trước khi thực hiện các bước xử lý tiếp theo. Chuyển ảnh xám giúp giảm số kênh xử lý từ ba kênh

xuống một kênh, làm giảm khối lượng tính toán và đơn giản hóa các bước như nhị phân hóa, phát hiện biên và tìm contour [2].

Một công thức chuyển ảnh RGB sang ảnh xám thường dùng là:

$$I(x, y) = 0,299R(x, y) + 0,587G(x, y) + 0,114B(x, y) \quad (2.1)$$

Trong đó $I(x, y)$ là giá trị mức xám tại điểm ảnh có tọa độ (x, y) , còn $R(x, y)$, $G(x, y)$, $B(x, y)$ lần lượt là cường độ của ba kênh màu tại điểm ảnh đó. Các hệ số trong công thức phản ánh mức độ nhạy khác nhau của thị giác con người đối với từng kênh màu.

2.1.2. Tiền xử lý và nhị phân hóa ảnh

Sau khi chuyển ảnh xám, ảnh đầu vào thường cần được tiền xử lý để giảm nhiễu và làm nổi bật vùng cần quan tâm. Trong thực tế, ảnh phương tiện có thể bị ảnh hưởng bởi ánh sáng yếu, bóng đổ, phản chiếu, rung camera hoặc nền phức tạp. Những yếu tố này làm cho bước xác định vùng biên số trở nên khó khăn hơn nếu xử lý trực tiếp trên ảnh gốc.

Một bước tiền xử lý thường dùng là làm mờ ảnh bằng bộ lọc trung bình hoặc Gaussian để giảm nhiễu cục bộ. Sau đó, ảnh được nhị phân hóa để tách các vùng sáng và tối. Nhị phân hóa ảnh là phép biến đổi ảnh xám thành ảnh chỉ có hai mức giá trị, thường là 0 và 255. Với một ngưỡng T cho trước, phép nhị phân hóa có thể được biểu diễn như sau:

$$g(x, y) = 255 \text{ nếu } I(x, y) \geq T; \quad g(x, y) = 0 \text{ nếu } I(x, y) < T \quad (2.2)$$

Trong đó $I(x, y)$ là ảnh xám đầu vào, còn $g(x, y)$ là ảnh nhị phân đầu ra. Khi ngưỡng T được chọn phù hợp, vùng chứa ký tự và khung biên số sẽ nổi bật hơn so với nền xung quanh. Tuy nhiên, nếu dùng một ngưỡng cố định cho toàn ảnh, kết quả có thể không ổn định trong điều kiện ánh sáng thay đổi.

Một phương pháp chọn ngưỡng tự động thường được sử dụng là phương pháp Otsu. Phương pháp này xem histogram mức xám của ảnh gồm hai lớp chính là nền và vật thể, sau đó tìm ngưỡng T sao cho phương sai giữa hai lớp đạt cực đại [3]. Ý tưởng này có thể viết dưới dạng:

$$t^* = \arg \max_{t \in [0, L-1]} \sigma_B^2(t) \quad (2.3)$$

$$\sigma_b^2(t) = \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2 \quad (2.4)$$

Trong đó ω_0 , ω_1 là xác suất xuất hiện của hai lớp điểm ảnh, còn μ_0 , μ_1 là mức xám trung bình của từng lớp. Nhờ chọn ngưỡng dựa trên phân bố mức xám của ảnh, Otsu thường ổn định hơn so với việc chọn ngưỡng thủ công trong các ảnh có độ tương phản rõ.

2.1.3. Phát hiện biên và tìm đường viền

Sau khi ảnh được tiền xử lý và nhị phân hóa, hệ thống cần xác định những vùng có khả năng là biển số. Một cách tiếp cận phổ biến là phát hiện các đường viền (contour). Contour là tập hợp các điểm biên liên thông tạo thành ranh giới của một vùng vật thể trên ảnh nhị phân. Trong bài toán biển số, vùng biển số thường có dạng gần hình chữ nhật và chứa nhiều cạnh rõ ràng, do đó có thể được phát hiện thông qua các contour có đặc trưng hình học phù hợp.

Thuật toán dò đường viền của Suzuki và Abe là cơ sở cho hàm tìm contour trong OpenCV. Thuật toán này quét ảnh nhị phân để phát hiện các đường biên ngoài và đường biên trong, đồng thời lưu lại quan hệ phân cấp giữa các contour [4]. Kết quả của bước tìm contour là tập hợp các vùng ứng viên, từ đó hệ thống tiếp tục lọc ra vùng có khả năng là biển số.

Để đánh giá một contour có phù hợp với biển số hay không, có thể dùng các đặc trưng hình học đơn giản như diện tích, hình chữ nhật bao quanh, tỉ lệ chiều rộng trên chiều cao và mức độ lấp đầy của contour trong hình chữ nhật bao quanh. Một số đại lượng thường dùng gồm:

$$Aspect\ ratio = w / h \quad (2.5)$$

$$Rectangularity = A_c / (w \times h) \quad (2.6)$$

Trong đó w và h là chiều rộng, chiều cao của hình chữ nhật bao quanh contour, còn A_c là diện tích contour. Biển số xe thường có dạng hình chữ nhật ngang, vì vậy tỉ lệ w/h thường nằm trong một khoảng nhất định. Những contour có diện tích quá nhỏ, quá lớn hoặc tỉ lệ hình học không phù hợp sẽ bị loại bỏ. Đây là cơ sở lý thuyết cho bước lọc contour và chọn vùng biển số trong thiết kế phần mềm ở Chương 3.

2.1.4. Vai trò của OpenCV trong hệ thống

OpenCV (Open Source Computer Vision Library) là thư viện mã nguồn mở cung cấp nhiều hàm xử lý ảnh và thị giác máy tính như chuyển đổi không gian màu, lọc ảnh, nhị phân hóa, tìm contour, biến đổi hình thái học và cắt vùng ảnh quan tâm [5]. Thay vì tự cài đặt lại các thuật toán nền tảng, hệ thống sử dụng OpenCV để xây dựng pipeline xử lý ảnh gồm các bước: thu ảnh từ camera, chuyển ảnh xám, tiền xử lý, nhị phân hóa, tìm contour, lọc vùng ứng viên và cắt ảnh biên số đưa sang OCR.

Ưu điểm của cách tiếp cận cổ điển bằng OpenCV là dễ triển khai, dễ giải thích, yêu cầu tài nguyên tính toán thấp và phù hợp với hệ thống chạy cục bộ. Tuy nhiên, phương pháp này nhạy với điều kiện ánh sáng, góc chụp, độ mờ ảnh và sự khác biệt về mẫu biên số. Do đó, chất lượng ảnh đầu vào và các ngưỡng lọc hình học có ảnh hưởng trực tiếp đến độ chính xác nhận diện.

2.2. NHẬN DẠNG KÝ TỰ QUANG HỌC VỚI TESSERACT OCR

2.2.1. Khái niệm OCR

Nhận dạng ký tự quang học (Optical Character Recognition - OCR) là quá trình chuyển đổi nội dung chữ xuất hiện trong ảnh thành chuỗi ký tự mà máy tính có thể xử lý. Trong các ứng dụng thông thường, OCR được dùng để nhận dạng văn bản từ tài liệu quét, ảnh chụp giấy tờ hoặc biểu mẫu. Trong hệ thống nhận diện biên số, OCR được áp dụng cho vùng ảnh biên số đã được cắt ra từ bước xử lý ảnh.

Quá trình OCR thường gồm các bước: tiền xử lý ảnh đầu vào, xác định vùng hoặc dòng văn bản, nhận dạng ký tự và hậu xử lý kết quả. Độ chính xác OCR phụ thuộc mạnh vào chất lượng ảnh đầu vào. Ảnh bị mờ, nghiêng, thiếu sáng, dư sáng hoặc có nhiễu nền sẽ làm tăng khả năng nhầm lẫn giữa các ký tự có hình dạng tương tự, ví dụ 0 và O, 1 và I, 5 và S, 8 và B.

2.2.2. Tesseract OCR

Tesseract là một công cụ OCR mã nguồn mở, ban đầu được phát triển tại Hewlett-Packard và sau đó được Google tiếp tục duy trì. Các phiên bản Tesseract

hiện nay hỗ trợ cơ chế nhận dạng dựa trên mạng nơ-ron LSTM, đồng thời vẫn duy trì khả năng sử dụng engine nhận dạng cũ trong một số trường hợp [6], [7]. Với hệ thống trong đồ án, Tesseract được sử dụng như một thành phần nhận dạng ký tự độc lập sau khi OpenCV đã xác định và cắt vùng biển số.

Khi dùng Tesseract, một số tham số cấu hình có ảnh hưởng trực tiếp đến kết quả nhận dạng. Page Segmentation Mode (PSM) quy định cách Tesseract giả định bố cục văn bản trong ảnh đầu vào. Với biển số xe, ảnh đầu vào thường chỉ chứa một dòng hoặc một vùng ký tự ngắn, do đó cần chọn PSM phù hợp để tránh việc Tesseract phân tích ảnh như một trang văn bản đầy đủ. OCR Engine Mode (OEM) quy định engine nhận dạng được sử dụng. Ngoài ra, Tesseract cho phép giới hạn tập ký tự hợp lệ thông qua danh sách ký tự cho phép, ví dụ chỉ gồm chữ in hoa và chữ số đối với biển số xe.

Việc giới hạn không gian nhận dạng là rất quan trọng trong bài toán biển số. Khác với văn bản tự nhiên, biển số không tạo thành các từ có nghĩa trong từ điển và không có ngữ cảnh ngôn ngữ để tự sửa lỗi. Vì vậy, nếu OCR nhận sai một ký tự, kết quả cuối cùng thường sai trực tiếp. Do đó, ngoài việc cải thiện ảnh đầu vào, hệ thống cần kết hợp cấu hình OCR và hậu xử lý chuỗi ký tự để tăng độ tin cậy.

2.2.3. Đặc thù OCR trong nhận diện biển số

OCR cho biển số xe có một số đặc điểm khác với OCR tài liệu thông thường. Thứ nhất, số lượng ký tự trên biển số ngắn, bố cục tương đối cố định và chủ yếu gồm chữ in hoa, chữ số. Đây là điểm thuận lợi vì phạm vi ký tự cần nhận dạng nhỏ. Thứ hai, ảnh biển số thường được chụp ngoài môi trường thực tế, chịu ảnh hưởng của ánh sáng, góc đặt camera, khoảng cách, bụi bẩn, phản chiếu và chuyển động của xe. Đây là điểm bất lợi vì vùng ký tự có thể bị biến dạng hoặc mất nét.

Trong hệ thống, kết quả OCR cần được hậu xử lý trước khi đưa vào cơ sở dữ liệu hoặc dùng để ra quyết định. Một số bước hậu xử lý thường gặp gồm loại bỏ khoảng trắng thừa, loại bỏ ký tự không hợp lệ, chuyển toàn bộ chữ cái về chữ in hoa và kiểm tra chuỗi nhận dạng theo định dạng biển số dự kiến. Nếu hệ thống

áp dụng cho biển số Việt Nam, có thể bổ sung quy tắc kiểm tra định dạng để loại bỏ các kết quả không hợp lệ hoặc yêu cầu nhận diện lại khi độ tin cậy thấp.

Như vậy, Tesseract không hoạt động độc lập mà phụ thuộc chặt chẽ vào bước tiền xử lý ảnh bằng OpenCV. Vùng ảnh biển số càng rõ, thẳng, ít nhiễu và có độ tương phản tốt thì xác suất nhận dạng đúng càng cao. Đây là cơ sở cho việc thiết kế quy trình tiền xử lý, cắt vùng ảnh và cấu hình OCR ở Chương 3.

2.3. VI ĐIỀU KHIỂN ARDUINO VÀ GIAO TIẾP SERIAL

2.3.1. Vi điều khiển Arduino trong hệ thống nhúng

Arduino là một nền tảng phát triển phần cứng mã nguồn mở, thường được dùng trong các hệ thống nhúng quy mô nhỏ nhờ dễ lập trình, có nhiều chân vào/ra và cộng đồng hỗ trợ rộng. Board Arduino sử dụng trong các hệ thống phổ biến như Arduino Uno hoặc Arduino Nano thường dựa trên vi điều khiển ATmega328P, một vi điều khiển 8-bit tích hợp bộ nhớ chương trình, bộ nhớ dữ liệu, bộ định thời, cổng vào/ra số, bộ chuyển đổi tương tự - số và giao tiếp nối tiếp UART [8], [9].

Chương trình Arduino thường gồm hai hàm chính: `setup()` và `loop()`. Hàm `setup()` được thực thi một lần khi board khởi động, dùng để cấu hình chân vào/ra, tốc độ Serial và các trạng thái ban đầu. Hàm `loop()` được thực thi lặp lại liên tục, dùng để đọc tín hiệu cảm biến, xử lý điều kiện và xuất tín hiệu điều khiển đến các thiết bị chấp hành. Mô hình lặp liên tục này phù hợp với hệ thống bãi xe vì trạng thái xe vào/ra, cảm biến và cổng chặn cần được kiểm tra thường xuyên.

2.3.2. Tín hiệu vào/ra số và thiết bị ngoại vi

Trong hệ thống bãi xe tự động, Arduino thường đóng vai trò trung gian giữa máy tính và các thiết bị phần cứng như cảm biến phát hiện xe, nút nhấn, đèn báo, còi báo hoặc cơ cấu đóng/mở cổng. Các chân vào/ra số của Arduino làm việc với hai mức logic: HIGH và LOW. Khi một cảm biến phát hiện có xe, tín hiệu trả về có thể thay đổi từ LOW sang HIGH hoặc ngược lại tùy loại cảm biến và cách đấu nối.

Việc đọc tín hiệu số cần xét đến hiện tượng nhiễu hoặc đọc lặp. Ví dụ, khi xe đứng tại vị trí cảm biến trong một khoảng thời gian, chương trình có thể đọc được nhiều lần cùng một trạng thái nếu không có cơ chế chống lặp. Vì vậy, phần mềm nhúng thường cần dùng biến trạng thái, thời gian chờ hoặc điều kiện chuyển trạng thái để đảm bảo một xe chỉ được ghi nhận một lần trong một chu kỳ xử lý.

Đối với thiết bị chấp hành như servo hoặc motor điều khiển công, Arduino xuất tín hiệu điều khiển theo lệnh nhận được từ máy tính hoặc theo trạng thái cảm biến. Khi biến số được nhận diện hợp lệ, máy tính có thể gửi lệnh mở công qua Serial; Arduino nhận lệnh, điều khiển cơ cấu mở công, sau đó cập nhật trạng thái cảm biến để xác định xe đã đi qua hay chưa.

2.3.3. Giao tiếp Serial UART

Để trao đổi dữ liệu giữa Arduino và máy tính, hệ thống sử dụng giao tiếp nối tiếp không đồng bộ theo chuẩn UART. Trong giao tiếp UART, dữ liệu được truyền từng bit theo thứ tự thời gian qua đường truyền nối tiếp. Mỗi khung dữ liệu thường gồm bit bắt đầu, các bit dữ liệu, tùy chọn bit kiểm tra chẵn lẻ và bit kết thúc. Hai thiết bị không dùng chung xung clock mà cần thống nhất trước tốc độ truyền, gọi là baud rate [10].

Trong môi trường Arduino, giao tiếp giữa máy tính và board thường được thực hiện thông qua cổng USB-to-Serial. Máy tính gửi chuỗi lệnh hoặc nhận chuỗi trạng thái từ Arduino dưới dạng các byte ký tự. Vì dữ liệu Serial là một luồng byte liên tục, bên gửi và bên nhận cần thống nhất quy ước đóng gói dữ liệu để tách đúng từng bản tin. Một cách đơn giản là dùng ký tự phân cách giữa các trường dữ liệu và ký tự xuống dòng để đánh dấu kết thúc bản tin.

Ví dụ, một bản tin trạng thái có thể được thiết kế theo dạng tổng quát:

MODE;SENSOR_IN;SENSOR_OUT;CURRENT_SLOT

Trong đó MODE biểu diễn chế độ hoạt động, SENSOR_IN và SENSOR_OUT biểu diễn trạng thái cảm biến vào/ra, còn CURRENT_SLOT biểu diễn số chỗ hiện tại. Khi nhận bản tin, chương trình máy tính tách chuỗi theo ký tự phân cách, kiểm tra số lượng trường và chuyển đổi dữ liệu về kiểu phù hợp. Cách đóng gói này đơn giản, dễ kiểm tra lỗi và phù hợp với hệ thống quy mô nhỏ.

2.4. KIẾN TRÚC MÁY CHỦ CỤC BỘ, FLASK VÀ SQLITE

2.4.1. Mô hình client-server cục bộ

Mô hình client-server là mô hình kiến trúc phần mềm trong đó máy khách gửi yêu cầu và máy chủ xử lý, sau đó trả về phản hồi. Trong hệ thống của đồ án, trình duyệt web đóng vai trò client, còn chương trình Python chạy trên máy tính đóng vai trò server cục bộ. Server này vừa cung cấp giao diện giám sát, vừa truy xuất cơ sở dữ liệu và cập nhật thông tin từ quá trình nhận diện biển số, giao tiếp Arduino.

Việc triển khai server cục bộ giúp hệ thống không phụ thuộc vào máy chủ từ xa hoặc kết nối Internet. Điều này phù hợp với mô hình bãi xe quy mô nhỏ, nơi toàn bộ xử lý nhận diện, lưu trữ và giám sát có thể thực hiện trên một máy tính đặt tại điểm kiểm soát. Tuy nhiên, do nhiều thành phần cùng chạy trên một máy, thiết kế phần mềm cần chú ý đến vấn đề chia sẻ tài nguyên, đặc biệt là truy cập cơ sở dữ liệu.

2.4.2. Flask và REST API

Flask là một micro-framework phát triển web bằng Python, cung cấp các thành phần cần thiết để xây dựng ứng dụng web như định tuyến URL, xử lý request và trả về response [11]. Khác với các framework full-stack, Flask có cấu trúc nhẹ, linh hoạt và phù hợp với các ứng dụng nhỏ hoặc các API nội bộ. Trong hệ thống bãi xe, Flask được dùng để xây dựng máy chủ cục bộ phục vụ giao diện giám sát và cung cấp dữ liệu cho trình duyệt.

Một route trong Flask ánh xạ một đường dẫn URL tới một hàm xử lý cụ thể. Hàm này có thể trả về trang HTML hoặc dữ liệu dạng JSON. Đối với giao diện giám sát, cách thường dùng là trang web tải giao diện ban đầu, sau đó định kỳ gửi request đến các API để lấy dữ liệu mới như số chỗ trống, danh sách xe đang gửi, lịch sử xe vào/ra hoặc dữ liệu dự đoán. Dữ liệu JSON có ưu điểm là gọn, dễ xử lý bằng JavaScript và phù hợp cho giao tiếp giữa client và server.

Mặc dù các API trong hệ thống có thể không phải là REST đầy đủ theo nghĩa chặt chẽ, cách tổ chức route theo tài nguyên và trả dữ liệu JSON giúp mã

nguồn rõ ràng, dễ mở rộng. Ví dụ, một route có thể phục vụ dữ liệu trạng thái bãi xe, một route khác phục vụ lịch sử xe vào/ra, và một route khác phục vụ kết quả dự đoán.

2.4.3. SQLite và mô hình dữ liệu quan hệ

SQLite là hệ quản trị cơ sở dữ liệu quan hệ dạng nhúng. Khác với các hệ quản trị cơ sở dữ liệu cần một tiến trình server riêng, SQLite lưu toàn bộ cơ sở dữ liệu trong một file và được truy cập trực tiếp thông qua thư viện trong ứng dụng [12]. Đặc điểm này phù hợp với hệ thống cục bộ quy mô nhỏ vì dễ cài đặt, không cần cấu hình máy chủ cơ sở dữ liệu và đủ đáp ứng nhu cầu lưu trữ lịch sử vào/ra, trạng thái chỗ trống và dữ liệu phục vụ dự đoán.

Dữ liệu trong SQLite được tổ chức theo mô hình quan hệ, gồm các bảng, dòng và cột. Mỗi bảng biểu diễn một nhóm thực thể hoặc sự kiện trong hệ thống. Đối với hệ thống bãi xe, cơ sở dữ liệu có thể gồm bảng lưu lịch sử xe vào/ra, bảng lưu thông tin xe đang trong bãi, bảng trạng thái số chỗ trống và bảng dữ liệu dùng cho mô hình dự đoán. Các bảng có thể dùng khóa chính để định danh bản ghi và khóa ngoại để liên kết dữ liệu khi cần [13].

Một yêu cầu quan trọng của cơ sở dữ liệu trong hệ thống bãi xe là tính nhất quán. Khi một xe vào hoặc ra, hệ thống không chỉ ghi lịch sử sự kiện mà còn cập nhật số chỗ trống hiện tại. Các thao tác này nên được thực hiện trong một giao dịch để tránh trường hợp ghi nhận lịch sử thành công nhưng cập nhật số chỗ thất bại, hoặc ngược lại. Đây là lý do các khái niệm giao dịch và nhất quán dữ liệu cần được xem xét trong thiết kế cơ sở dữ liệu ở Chương 3.

2.4.4. Đa luồng và cơ chế cập nhật dữ liệu bằng polling

Trong hệ thống, chương trình chính có thể đồng thời thực hiện nhiều nhiệm vụ: nhận ảnh từ camera, xử lý nhận diện biển số, giao tiếp Arduino, cập nhật cơ sở dữ liệu và phục vụ giao diện web. Để các nhiệm vụ này không chặn lẫn nhau, có thể sử dụng mô hình đa luồng. Ví dụ, máy chủ Flask có thể chạy trên một luồng riêng, trong khi luồng chính xử lý giao diện điều khiển hoặc nhận diện biển số.

Khi nhiều luồng cùng truy cập cơ sở dữ liệu, cần có cơ chế tránh xung đột. Với SQLite, một cách triển khai an toàn là mỗi luồng sử dụng kết nối cơ sở dữ liệu riêng và các thao tác ghi dữ liệu được thực hiện ngắn gọn trong giao dịch. Nếu hệ thống có nhiều thao tác ghi đồng thời, cần bổ sung khóa đồng bộ hoặc hàng đợi ghi dữ liệu để tránh lỗi tranh chấp tài nguyên.

Đối với giao diện giám sát, hệ thống sử dụng cơ chế polling: trình duyệt định kỳ gửi request đến server để lấy dữ liệu mới. Polling đơn giản hơn so với WebSocket vì không cần duy trì kết nối hai chiều liên tục. Với hệ thống bãi xe quy mô nhỏ, polling đủ đáp ứng yêu cầu cập nhật trạng thái nếu chọn chu kỳ hợp lý. Các dữ liệu cần cập nhật nhanh, như số chỗ trống, có thể polling với chu kỳ ngắn hơn; dữ liệu ít thay đổi hơn, như biểu đồ dự đoán, có thể polling với chu kỳ dài hơn.

2.5. MÔ HÌNH DỰ ĐOÁN SỐ CHỖ TRỐNG SEASONAL-MEAN

2.5.1. Chuỗi thời gian và tính mùa vụ

Dự báo chuỗi thời gian là bài toán ước lượng giá trị tương lai của một đại lượng dựa trên các quan sát trong quá khứ theo thứ tự thời gian. Một chuỗi thời gian có thể chứa các thành phần như xu hướng dài hạn, dao động ngẫu nhiên và tính mùa vụ. Tính mùa vụ là hiện tượng dữ liệu lặp lại theo một chu kỳ nhất định, ví dụ theo giờ trong ngày, ngày trong tuần hoặc tháng trong năm [14].

Số chỗ trống trong bãi xe là một đại lượng có khả năng mang tính mùa vụ. Trong thực tế, nhu cầu gửi xe thường thay đổi theo giờ học, giờ làm, giờ tan ca hoặc theo từng ngày trong tuần. Ví dụ, số xe vào bãi có thể tăng mạnh vào buổi sáng và giảm vào cuối ngày. Do đó, dữ liệu lịch sử số chỗ trống có thể được khai thác để ước lượng số chỗ trống tại các thời điểm tương lai có cùng đặc điểm thời gian.

2.5.2. Mô hình Seasonal-Mean

Seasonal-Mean là mô hình dự đoán đơn giản dựa trên giả định rằng các thời điểm có cùng đặc điểm mùa vụ sẽ có giá trị trung bình gần nhau. Trong hệ thống

này, hai đặc điểm mùa vụ được xét là giờ trong ngày và thứ trong tuần. Với một thời điểm cần dự đoán có giờ h và thứ d , mô hình lấy trung bình các giá trị số chỗ trống đã quan sát trong lịch sử tại đúng cặp thời gian (h, d) .

Gọi y_t là số chỗ trống quan sát được tại thời điểm t . Hàm $\text{hour}(t)$ trả về giờ trong ngày của thời điểm t , còn $\text{dow}(t)$ trả về thứ trong tuần. Giá trị dự đoán tại giờ h và thứ d được xác định như sau:

$$\hat{y}(h, d) = \text{mean} \{ y_t \mid \text{hour}(t) = h \text{ và } \text{dow}(t) = d \} \quad (2.7)$$

Trong đó $\hat{y}(h, d)$ là số chỗ trống dự đoán. Nếu tại một cặp (h, d) chưa có đủ dữ liệu lịch sử, hệ thống có thể dùng phương án dự phòng như trung bình theo cùng giờ h trên toàn bộ các ngày, trung bình theo cùng thứ d hoặc trung bình toàn bộ dữ liệu. Cách xử lý này giúp mô hình vẫn đưa ra được dự đoán trong giai đoạn dữ liệu còn ít.

Mô hình Seasonal-Mean có thể xem là một biến thể của mô hình seasonal naïve. Seasonal naïve dự đoán giá trị tương lai bằng giá trị cùng mùa vụ ở chu kỳ gần nhất, trong khi Seasonal-Mean lấy trung bình nhiều quan sát cùng mùa vụ trong lịch sử. Việc lấy trung bình giúp làm mượt nhiễu và giảm phụ thuộc vào một quan sát đơn lẻ [14].

Ưu điểm của Seasonal-Mean là dễ hiểu, dễ triển khai, không cần huấn luyện phức tạp và phù hợp làm mô hình nền cho hệ thống quy mô nhỏ. Tuy nhiên, mô hình này không học được xu hướng thay đổi dài hạn và phản ứng kém với các sự kiện bất thường như ngày lễ, thời tiết xấu, sự kiện đông người hoặc thay đổi quy mô bãi xe. Vì vậy, khi trình bày kết quả ở Chương 4, cần nêu rõ đây là mô hình dự đoán đơn giản dựa trên dữ liệu lịch sử, không phải mô hình học máy phức tạp.

2.5.3. Đánh giá mô hình dự đoán

Để đánh giá mô hình dự đoán số chỗ trống, cần so sánh giá trị dự đoán với giá trị thực tế trên một tập kiểm thử. Hai chỉ số sai số phổ biến cho bài toán dự báo định lượng là MAE và RMSE [15].

Sai số tuyệt đối trung bình (Mean Absolute Error - MAE) được tính bằng trung bình giá trị tuyệt đối của sai số:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (2.8)$$

Trong đó y_i là giá trị thực tế, $\hat{y}_i$ là giá trị dự đoán và n là số quan sát trong tập kiểm thử. MAE có cùng đơn vị với đại lượng dự đoán nên dễ diễn giải. Ví dụ, MAE bằng 2 nghĩa là trung bình mô hình dự đoán lệch khoảng 2 chỗ so với thực tế.

Sai số căn bậc hai trung bình bình phương (Root Mean Squared Error - RMSE) được tính như sau:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2.9)$$

Do sai số được bình phương trước khi lấy trung bình, RMSE nhạy hơn MAE đối với các trường hợp dự đoán sai nhiều. Vì vậy, khi đánh giá mô hình, MAE giúp phản ánh sai số trung bình dễ hiểu, còn RMSE giúp phát hiện các sai số lớn có thể ảnh hưởng đến trải nghiệm sử dụng hệ thống.

2.5.4. Chia dữ liệu huấn luyện và kiểm thử theo thời gian

Đối với dữ liệu chuỗi thời gian, cách chia dữ liệu có ảnh hưởng lớn đến độ tin cậy của kết quả đánh giá. Không nên chia ngẫu nhiên dữ liệu như các bài toán học máy thông thường vì điều này có thể làm dữ liệu tương lai xuất hiện trong tập huấn luyện, gây rò rỉ dữ liệu và làm kết quả đánh giá trở nên lạc quan giả tạo.

Cách phù hợp hơn là chia theo thời gian: dữ liệu cũ hơn được dùng để xây dựng mô hình, còn dữ liệu mới hơn được dùng để kiểm thử. Cách đánh giá này phản ánh đúng tình huống triển khai thực tế, trong đó hệ thống chỉ có thể sử dụng dữ liệu đã xảy ra để dự đoán cho tương lai [16]. Đây là cơ sở cho việc đánh giá mô hình Seasonal-Mean ở phần thực nghiệm.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1. YÊU CẦU VÀ SƠ ĐỒ KHỐI HỆ THỐNG

3.1.1. Yêu cầu hệ thống

Trên cơ sở khảo sát, nghiên cứu thực trạng quản lý bãi đỗ xe tại các khu vực có quy mô nhỏ và vừa, hệ thống được thiết kế nhằm hỗ trợ tự động hóa quá trình giám sát, kiểm soát phương tiện ra/vào và quản lý trạng thái các ô đỗ xe. Các yêu cầu của hệ thống được chia thành hai nhóm chính: yêu cầu chức năng và yêu cầu phi chức năng.

Yêu cầu chức năng:

- Phát hiện phương tiện tại khu vực cổng vào và cổng ra thông qua cảm biến hồng ngoại, làm cơ sở kích hoạt quá trình xử lý ra/vào bãi đỗ.
- Cho phép xác thực phương tiện bằng hai yếu tố gồm mã thẻ RFID và biển số xe được nhận dạng từ hình ảnh camera, nhằm nâng cao độ tin cậy trong quá trình kiểm so
- Giám sát trạng thái còn trống/đã có xe của từng ô đỗ theo thời gian thực bằng cảm biến hồng ngoại, phục vụ việc hiển thị tại hiện trường và trên giao diện web.
- Điều khiển tự động hai thanh chắn (barrier) ra vào bằng động cơ servo; đồng thời cho phép vận hành viên chuyển sang chế độ điều khiển bằng tay khi cần thiết.
- Đối chiếu cặp dữ liệu thẻ RFID và biển số xe đã lưu tại thời điểm xe vào với dữ liệu đọc được tại thời điểm xe ra, chỉ cho phép mở thanh chắn khi cả hai yếu tố trùng khớp, nhằm hạn chế rủi ro nhầm xe hoặc mất xe.
- Tính thời gian gửi xe và phí gửi xe dựa trên thời điểm xe vào và thời điểm xe ra.

- Lưu trữ toàn bộ lịch sử hoạt động (sự kiện xe vào/ra, thay đổi trạng thái từng ô đỗ) vào cơ sở dữ liệu cục bộ, làm nền cho việc thống kê và dự đoán.
- Cung cấp giao diện web (dashboard) hiển thị trạng thái bãi đỗ, lịch sử ra/vào và các biểu đồ thống kê, tự động cập nhật theo thời gian thực mà không cần người dùng làm mới trang.
- Dự đoán số chỗ trống dự kiến cho 24 giờ tiếp theo dựa trên dữ liệu lịch sử đã được ghi nhận, hỗ trợ người quản lý chủ động bố trí vận hành.
- Phát tín hiệu cảnh báo bằng còi trong các trường hợp như bãi đỗ đã đầy, thông tin xác thực không hợp lệ hoặc có sự cố trong quá trình kiểm soát xe ra/vào.

Yêu cầu phi chức năng:

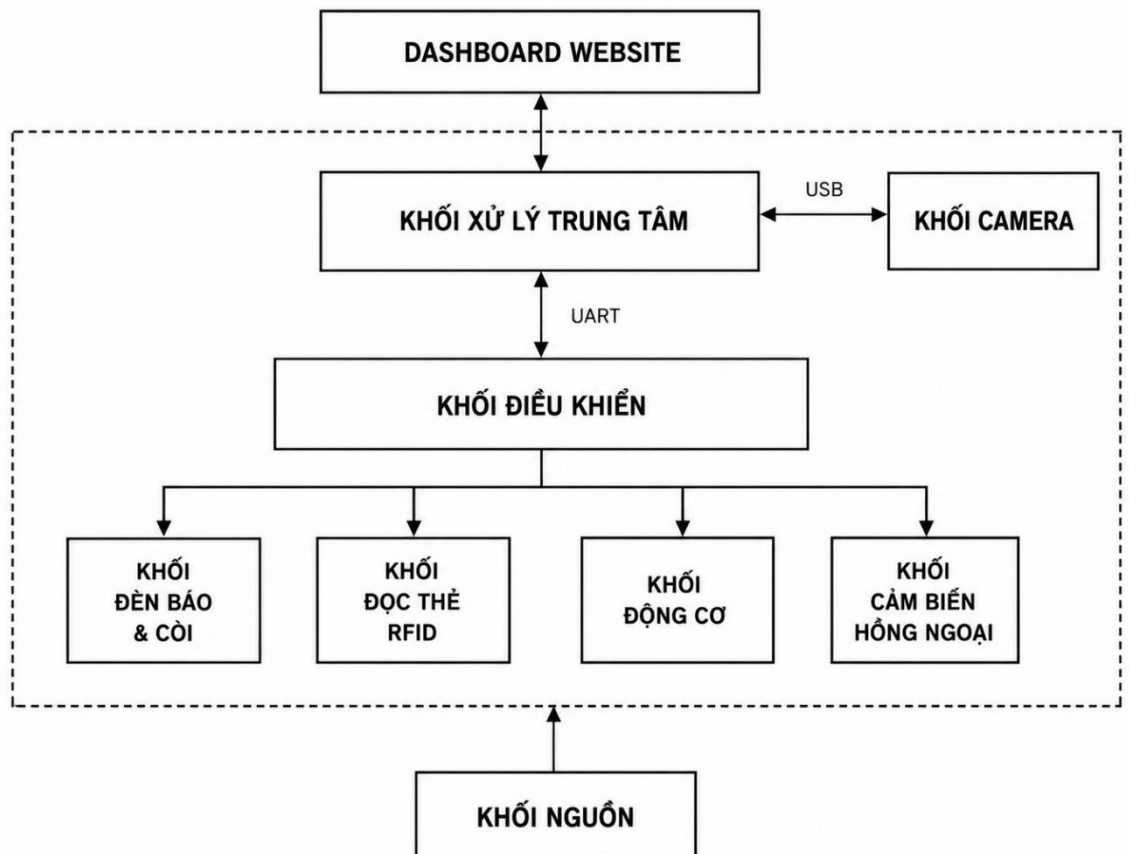
- Hệ thống hoạt động ổn định, cục bộ tại hiện trường, không bắt buộc phải có kết nối Internet để duy trì các chức năng giám sát và điều khiển cốt lõi.
- Thời gian phản hồi của các thao tác điều khiển cơ bản như đọc cảm biến, cập nhật trạng thái và đóng/mở thanh chắn cần đủ nhanh để không gây ảnh hưởng đến quá trình xe ra/vào. Chu kỳ truyền dữ liệu giữa Arduino và máy tính được thiết kế ở mức 200 ms.
- Chi phí triển khai phần cứng thấp, linh kiện phổ biến, dễ thay thế, phù hợp với quy mô mô hình thực nghiệm mô phỏng 3 ô đỗ.
- Kiến trúc phần mềm có khả năng mở rộng số lượng ô đỗ và nâng cấp mô hình dự đoán trong tương lai mà không phải thiết kế lại toàn bộ hệ thống.
- Hệ thống cần đảm bảo tính an toàn trong vận hành: thanh chắn không được mở khi bãi đã đầy hoặc khi thông tin xác thực giữa thẻ RFID và biển số xe không trùng khớp.

3.1.2. Sơ đồ khối và chức năng của từng khối

Dựa trên các yêu cầu đã xác định ở mục 3.1.1, hệ thống được phân chia thành các khối chức năng nhằm bảo đảm quá trình thu nhận dữ liệu, xử lý thông tin, điều khiển thiết bị và giám sát vận hành được thực hiện một cách rõ ràng. Việc

phân chia theo khối giúp xác định vai trò của từng thành phần trong hệ thống, đồng thời thuận tiện cho quá trình thiết kế, lắp đặt, kiểm thử và mở rộng sau này.

Hệ thống bao gồm các khối chức năng như khối camera, khối hiển thị biểu đồ được kết nối với khối xử lý trung tâm và khối xử lý ngoại vi để điều khiển khối động cơ mở cửa, khối đọc RFID, khối đèn và còi để cảnh báo. Cuối cùng là khối nguồn để cấp điện cho toàn bộ hệ thống



Hình 3.1: Sơ đồ khối tổng thể của hệ thống

Chức năng của từng khối:

- Khối xử lý trung tâm là thành phần đảm nhận các tác vụ xử lý chính của hệ thống. Khối này nhận dữ liệu từ khối điều khiển và hình ảnh từ khối camera, sau đó thực hiện nhận dạng biển số, đối chiếu thông tin thẻ RFID với biển số xe, xử lý logic xe vào/ra, tính phí gửi xe và cập nhật dữ liệu vào hệ thống

lưu trữ. Bên cạnh đó, khối xử lý trung tâm còn cung cấp dữ liệu cho giao diện dashboard website để người vận hành có thể theo dõi trạng thái bãi đỗ.

- Khối điều khiển đóng vai trò trung gian giữa các thiết bị phần cứng tại hiện trường và khối xử lý trung tâm. Khối này tiếp nhận tín hiệu từ cảm biến hồng ngoại và đầu đọc RFID, đồng thời điều khiển các thiết bị đầu ra như động cơ servo, đèn báo và còi cảnh báo. Ngoài ra, khối điều khiển còn thực hiện truyền dữ liệu trạng thái lên khối xử lý trung tâm thông qua giao tiếp UART.
- Khối cảm biến hồng ngoại có nhiệm vụ thu nhận tín hiệu từ môi trường để phát hiện sự xuất hiện của phương tiện. Trong hệ thống, các cảm biến hồng ngoại được bố trí tại khu vực cổng vào, cổng ra và các ô đỗ xe. Tín hiệu từ khối cảm biến được đưa về khối điều khiển để xác định phương tiện đang chờ tại cổng hoặc trạng thái trống/có xe của từng ô đỗ.
- Khối đọc thẻ RFID được sử dụng để đọc mã định danh của thẻ xe. Mã thẻ RFID là một trong hai thông tin dùng để xác thực phương tiện trong quá trình xe vào và xe ra khỏi bãi. Khi người dùng quét thẻ tại cổng vào hoặc cổng ra, dữ liệu thẻ được gửi về khối điều khiển, sau đó truyền lên khối xử lý trung tâm để phục vụ việc lưu trữ và đối chiếu thông tin.
- Khối camera có nhiệm vụ thu nhận hình ảnh phương tiện tại khu vực cổng vào và cổng ra. Hình ảnh từ camera được truyền trực tiếp về khối xử lý trung tâm thông qua kết nối USB. Dữ liệu hình ảnh này được sử dụng cho quá trình xử lý ảnh và nhận dạng biển số xe, từ đó tạo thêm một yếu tố xác thực bên cạnh mã thẻ RFID.
- Khối động cơ có nhiệm vụ điều khiển cơ cấu thanh chắn tại cổng vào và cổng ra. Khi phương tiện được xác thực hợp lệ hoặc đủ điều kiện ra/vào bãi, khối điều khiển sẽ phát tín hiệu điều khiển động cơ servo để mở thanh chắn. Sau khi phương tiện đi qua, thanh chắn được điều khiển đóng lại nhằm bảo đảm quá trình kiểm soát phương tiện diễn ra tự động.

- Khối đèn báo và còi được sử dụng để phản hồi trạng thái hoạt động của hệ thống tại hiện trường. Đèn báo giúp hiển thị trực quan trạng thái của các ô đỗ hoặc trạng thái vận hành, trong khi còi cảnh báo được kích hoạt trong các trường hợp bất thường như bãi đã đầy, xác thực không hợp lệ.
- Dashboard website là giao diện giám sát của hệ thống, cho phép người vận hành theo dõi trạng thái bãi đỗ thông qua trình duyệt. Giao diện này hiển thị các thông tin như số chỗ trống, trạng thái từng ô đỗ, lịch sử xe vào/ra, thống kê và kết quả dự đoán. Dữ liệu trên dashboard được lấy từ khối xử lý trung tâm và cập nhật theo thời gian gần thực.
- Khối nguồn có nhiệm vụ cung cấp điện áp và dòng điện phù hợp cho toàn bộ các khối phần cứng trong hệ thống. Việc thiết kế khối nguồn cần bảo đảm các thiết bị như khối điều khiển, cảm biến, RFID, camera, động cơ, đèn báo và còi hoạt động ổn định trong quá trình vận hành.

3.1.3. Mô tả hoạt động của hệ thống

Hệ thống vận hành theo hai chế độ: tự động (Auto) và thủ công (Manual), được lựa chọn thông qua bảng điều khiển. Ở chế độ thủ công, người giám sát chủ động đóng/mở từng thanh chắn thông qua giao diện điều khiển, dùng cho các trường hợp cần can thiệp thủ công. Ở chế độ tự động, quá trình phát hiện phương tiện, xác thực thông tin, điều khiển thanh chắn và cập nhật dữ liệu được thực hiện dựa trên tín hiệu từ cảm biến, thẻ RFID, camera và phần mềm xử lý trung tâm; song song với hai luồng này là luồng cập nhật trạng thái ô đỗ và truyền dữ liệu lên giao diện giám sát, diễn ra liên tục không phụ thuộc vào việc có xe ra/vào hay không.

Về tổng thể, hoạt động của hệ thống gồm ba luồng chính: luồng xử lý xe vào bãi, luồng xử lý xe ra khỏi bãi và luồng cập nhật trạng thái ô đỗ. Các luồng này có quan hệ phối hợp với nhau nhằm bảo đảm quá trình kiểm soát phương tiện được thực hiện tự động, đồng thời dữ liệu trạng thái của bãi đỗ luôn được cập nhật phục vụ giám sát và lưu trữ.

Luồng xử lý xe vào bãi đỗ:

- Khi phương tiện di chuyển đến khu vực cổng vào, cảm biến hồng ngoại tại cổng có nhiệm vụ phát hiện sự xuất hiện của xe và gửi tín hiệu về khối điều khiển. Sau khi phát hiện phương tiện, người dùng tiến hành quét thẻ RFID tại đầu đọc ở cổng vào. Đồng thời, camera tại cổng vào thu nhận hình ảnh phương tiện và gửi về khối xử lý trung tâm để thực hiện nhận dạng biển số xe.
- Khối xử lý trung tâm tiếp nhận dữ liệu mã thẻ RFID và kết quả nhận dạng biển số, sau đó kiểm tra trạng thái hiện tại của bãi đỗ. Nếu bãi còn chỗ trống và dữ liệu xác thực hợp lệ, hệ thống ghi nhận thông tin xe vào, bao gồm mã thẻ RFID, biển số xe và thời điểm vào bãi. Sau đó, khối điều khiển phát tín hiệu điều khiển động cơ servo mở thanh chắn để phương tiện đi vào.
- Trong trường hợp bãi đỗ đã đầy, hệ thống không cho phép mở thanh chắn và kích hoạt còi cảnh báo để thông báo cho người vận hành hoặc người dùng. Nếu hệ thống không đọc được thẻ RFID hoặc không nhận dạng được biển số xe, quá trình xác thực sẽ không được chấp nhận và người dùng cần thực hiện lại thao tác.

Luồng xử lý xe ra khỏi bãi đỗ:

- Khi phương tiện di chuyển đến khu vực cổng ra, cảm biến hồng ngoại tại cổng ra phát hiện sự xuất hiện của xe và gửi tín hiệu về khối điều khiển. Người dùng tiến hành quét lại thẻ RFID tại đầu đọc ở cổng ra, đồng thời camera cổng ra thu nhận hình ảnh biển số xe để khối xử lý trung tâm thực hiện nhận dạng.
- Sau khi nhận được mã thẻ RFID và biển số xe tại thời điểm xe ra, khối xử lý trung tâm tiến hành đối chiếu với dữ liệu đã được lưu khi xe vào bãi. Nếu mã thẻ RFID và biển số xe trùng khớp với thông tin đã lưu, hệ thống xác định phương tiện hợp lệ. Khi đó, hệ thống tính toán thời gian gửi xe dựa trên thời điểm vào và thời điểm ra, đồng thời tính phí gửi xe theo quy định đã thiết lập.

- Sau khi xác thực thành công, khối điều khiển phát tín hiệu điều khiển động cơ servo mở thanh chắn tại cổng ra để phương tiện rời khỏi bãi. Khi xe đi qua khỏi khu vực cổng ra, thanh chắn được đóng lại. Đồng thời, cảm biến tại ô đỗ tương ứng cập nhật trạng thái từ đã có xe sang trống và dữ liệu này được truyền lên khối xử lý trung tâm để lưu trữ và hiển thị trên giao diện giám sát.
- Trong trường hợp mã thẻ RFID hoặc biển số xe không trùng khớp với dữ liệu đã lưu, hệ thống không mở thanh chắn và kích hoạt còi cảnh báo. Tình huống này giúp hạn chế rủi ro nhầm xe, sai thẻ hoặc phương tiện không hợp lệ rời khỏi bãi. Người vận hành có thể kiểm tra lại thông tin và xử lý bằng chế độ điều khiển bằng tay nếu cần thiết.

Luồng cập nhật trạng thái các ô đỗ xe và dữ liệu giám sát:

- Bên cạnh hai luồng xử lý xe vào và xe ra, hệ thống còn thực hiện việc cập nhật trạng thái các ô đỗ theo chu kỳ. Khối điều khiển liên tục đọc tín hiệu từ các cảm biến hồng ngoại được bố trí tại từng ô đỗ để xác định ô đang trống hay đã có xe. Các dữ liệu trạng thái này được đóng gói và truyền lên khối xử lý trung tâm thông qua giao tiếp UART.
- Tại khối xử lý trung tâm, dữ liệu nhận được từ khối điều khiển được kiểm tra và so sánh với trạng thái trước đó. Khi phát hiện có sự thay đổi, hệ thống cập nhật trạng thái ô đỗ, ghi nhận lịch sử thay đổi vào cơ sở dữ liệu và đồng thời cung cấp dữ liệu mới cho dashboard website. Nhờ đó, người vận hành có thể theo dõi tình trạng bãi đỗ, số chỗ trống và lịch sử xe vào/ra theo thời gian gần thực mà không cần thao tác tải lại trang.
- Quá trình cập nhật trạng thái được thực hiện độc lập với quá trình xử lý xe vào và xe ra. Điều này giúp hệ thống duy trì khả năng giám sát liên tục, đồng thời bảo đảm dữ liệu lưu trữ phản ánh đúng tình trạng thực tế của bãi đỗ trong quá trình vận hành.

3.2. THIẾT KẾ PHẦN CỨNG

Phần cứng của hệ thống được tổ chức thành tám khối chức năng: khối xử lý trung tâm, khối điều khiển, khối đọc thẻ RFID, khối động cơ, khối camera, khối còi báo, khối cảm biến hồng ngoại và khối nguồn. Các thông số kỹ thuật trình bày dưới đây được tổng hợp từ tài liệu linh kiện và thiết kế mạch in (PCB) đã thực hiện cho hệ thống.

3.2.1. Khối xử lý trung tâm

Khối xử lý trung tâm là thành phần đảm nhận các tác vụ xử lý chính của hệ thống quản lý bãi đỗ xe. Trong đề tài này, máy tính được sử dụng làm khối xử lý trung tâm do có khả năng xử lý ảnh, lưu trữ dữ liệu và vận hành giao diện giám sát tốt hơn. Việc sử dụng máy tính giúp hệ thống có thể thực hiện đồng thời nhiều nhiệm vụ như nhận dữ liệu từ Arduino, xử lý hình ảnh từ camera, nhận dạng biển số xe, đối chiếu thông tin phương tiện, tính phí gửi xe, lưu trữ dữ liệu và cung cấp giao diện giám sát cho người vận hành.

Máy tính kết nối với khối điều khiển Arduino thông qua giao tiếp UART qua cổng USB. Thông qua kết nối này, máy tính nhận các dữ liệu trạng thái từ Arduino như tín hiệu cảm biến, mã thẻ RFID, trạng thái thanh chắn và trạng thái các ô đỗ. Đồng thời, máy tính cũng có thể gửi các tín hiệu điều khiển cần thiết xuống Arduino để phục vụ quá trình vận hành hệ thống.

Bên cạnh đó, máy tính được kết nối trực tiếp với camera tại cổng vào và cổng ra thông qua cổng USB. Hình ảnh thu nhận từ camera được đưa vào chương trình xử lý ảnh để xác định vùng biển số và nhận dạng ký tự trên biển số xe. Kết quả nhận dạng được sử dụng kết hợp với mã thẻ RFID nhằm tạo thành cơ sở xác thực phương tiện khi xe vào và khi xe ra khỏi bãi.

Tại khối xử lý trung tâm, phần mềm hệ thống được xây dựng bằng ngôn ngữ Python. Các thư viện và công cụ chính được sử dụng bao gồm OpenCV để xử lý ảnh, Tesseract OCR để nhận dạng ký tự biển số, SQLite để lưu trữ dữ liệu cục bộ, PyQt5 để xây dựng giao diện vận hành tại máy tính và Flask để cung cấp dữ

liệu cho dashboard website. Trong đó, Flask được triển khai như một thành phần phần mềm chạy trực tiếp trên máy tính trung tâm, không phải là một khối phần cứng độc lập.

Các dữ liệu được xử lý tại máy tính bao gồm thông tin xe vào, xe ra, biển số xe, mã thẻ RFID, thời điểm ra/vào, thời lượng gửi xe, phí gửi xe và trạng thái các ô đỗ. Những dữ liệu này được lưu vào cơ sở dữ liệu SQLite để phục vụ việc truy xuất lịch sử, thống kê và dự đoán số chỗ trống trong các khoảng thời gian tiếp theo.

Để bảo đảm hệ thống hoạt động ổn định trong quá trình thực nghiệm, máy tính cần đáp ứng một số yêu cầu cơ bản như hỗ trợ hệ điều hành Windows hoặc Linux, cài đặt được môi trường Python 3, có đủ cổng USB để kết nối với Arduino và camera, đồng thời có bộ xử lý và dung lượng bộ nhớ đủ để chạy đồng thời chương trình xử lý ảnh, giao diện điều khiển và dashboard website. Trong mô hình thực nghiệm, máy tính đóng vai trò là bộ xử lý cục bộ của toàn hệ thống, giúp các chức năng cốt lõi vẫn có thể hoạt động mà không phụ thuộc bắt buộc vào kết nối Internet.

3.2.2. Khối điều khiển thiết bị ngoại vi

Khối điều khiển thiết bị ngoại vi là thành phần trung gian giữa các thiết bị phần cứng tại hiện trường và khối xử lý trung tâm. Trong hệ thống này, Arduino Nano được sử dụng làm bộ điều khiển chính cho tầng thiết bị ngoại vi do có kích thước nhỏ gọn, dễ lập trình, số lượng chân vào/ra phù hợp với mô hình thực nghiệm và hỗ trợ giao tiếp nối tiếp với máy tính thông qua cổng USB.

Nhiệm vụ chính của khối điều khiển là thu nhận tín hiệu từ các cảm biến hồng ngoại và đầu đọc thẻ RFID, sau đó điều khiển các thiết bị đầu ra như động cơ servo, đèn báo trạng thái và còi cảnh báo. Khối này không thực hiện các tác vụ xử lý phức tạp như nhận dạng biển số, lưu trữ cơ sở dữ liệu hoặc hiển thị dashboard. Các tác vụ này được đảm nhận bởi khối xử lý trung tâm. Nhờ cách phân chia này, Arduino tập trung vào các thao tác điều khiển mức thấp, trong khi máy tính đảm nhận các tác vụ có khối lượng tính toán lớn hơn.

Về tín hiệu đầu vào, Arduino nhận dữ liệu từ các cảm biến hồng ngoại được bố trí tại cổng vào, cổng ra và các ô đỗ xe. Các cảm biến tại cổng có nhiệm vụ phát hiện phương tiện đang chờ trước thanh chắn, còn các cảm biến tại ô đỗ dùng để xác định trạng thái trống hoặc đã có xe của từng vị trí đỗ. Ngoài ra, Arduino còn kết nối với hai module RFID RC522 để đọc mã UID của thẻ tại cổng vào và cổng ra, phục vụ quá trình xác thực phương tiện.

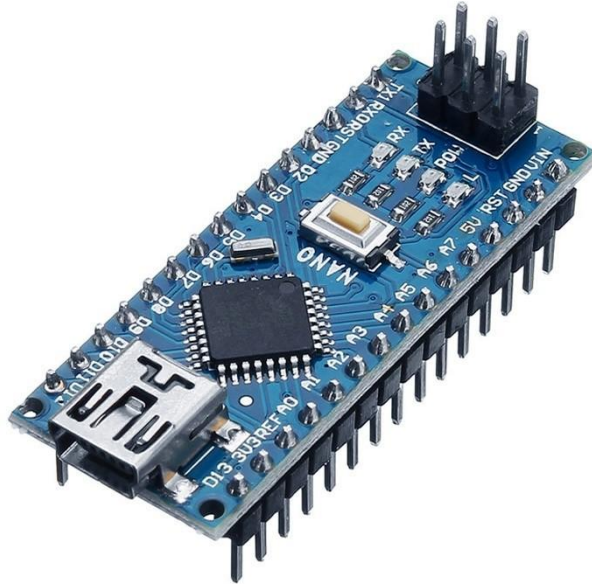
Về tín hiệu đầu ra, Arduino điều khiển hai động cơ servo dùng để đóng/mở thanh chắn tại cổng vào và cổng ra. Bên cạnh đó, Arduino còn điều khiển các đèn báo trạng thái ô đỗ và còi cảnh báo. Đèn báo giúp hiển thị trực quan trạng thái của từng ô đỗ tại mô hình, trong khi còi được kích hoạt trong các trường hợp như bãi đỗ đã đầy hoặc thông tin xác thực xe ra không hợp lệ.

Arduino giao tiếp với khối xử lý trung tâm thông qua chuẩn UART qua cổng USB. Dữ liệu từ Arduino được đóng gói thành chuỗi có cấu trúc cố định và truyền lên máy tính theo chu kỳ. Chuỗi dữ liệu bao gồm các thông tin về chế độ vận hành, trạng thái cảm biến, trạng thái xe vào/ra, số xe trong bãi và trạng thái của từng ô đỗ. Việc truyền dữ liệu định kỳ giúp khối xử lý trung tâm luôn nhận được trạng thái mới nhất của hệ thống để phục vụ lưu trữ, xử lý nghiệp vụ và cập nhật giao diện giám sát.

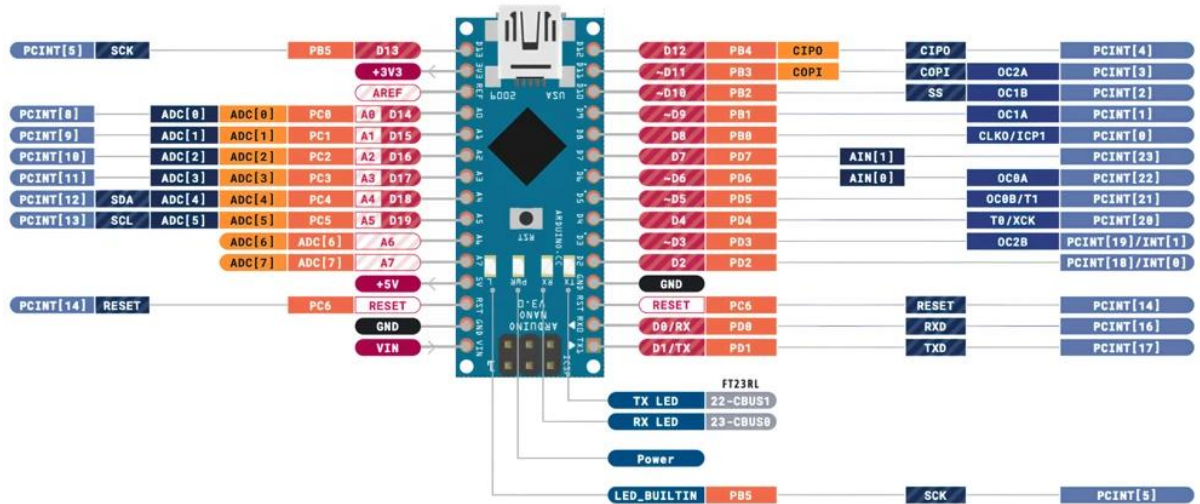
Trong chương trình điều khiển, các cảm biến tại ô đỗ được đọc và mã hóa thành các giá trị trạng thái. Giá trị 1 biểu thị ô đã có xe, còn giá trị 0 biểu thị ô còn trống. Các giá trị này được truyền lên máy tính cùng với các trường dữ liệu khác để cập nhật trạng thái bãi đỗ. Ngoài quá trình gửi dữ liệu định kỳ, Arduino cũng gửi dữ liệu khi xảy ra các sự kiện quan trọng như quét thẻ RFID, xe vào hoặc xe ra, nhằm bảo đảm hệ thống phản hồi kịp thời.

Khi thiết kế phần cứng cho khối điều khiển, cần chú ý đến vấn đề cấp nguồn và mức điện áp giao tiếp. Các thiết bị như servo, còi hoặc relay có dòng tiêu thụ lớn hơn khả năng cấp dòng trực tiếp từ chân Arduino, vì vậy cần được cấp nguồn từ khối nguồn riêng hoặc thông qua mạch điều khiển trung gian. Tuy nhiên, các khối này cần nối chung mass với Arduino để bảo đảm tín hiệu điều khiển hoạt

động chính xác. Đối với module RFID RC522, do module sử dụng điện áp 3,3V nên khi kết nối với Arduino cần bảo đảm mức điện áp tín hiệu phù hợp, tránh làm ảnh hưởng đến độ ổn định và tuổi thọ của module.



Hình 3.2: Arduino Nano



STT	Thông số	Chi tiết
1	Vi điều khiển chính	ATmega328P-AU
2	Điện áp cấp	5VDC cổng USB hoặc 6-9VDC chân Raw
3	Mức điện áp giao tiếp GPIO	TTL 5VDC
4	Dòng GPIO	40mA
5	Số chân Digital	14 chân, trong đó có 6 chân PWM
6	Số chân Analog	8 chân (hơn Arduino Uno 2 chân)
7	IC nạp và giao tiếp	USB UART FT232RL
8	Flash Memory	32KB (2KB Bootloader).
9	SRAM	2KB
10	EEPROM	1KB
11	Clock Speed	16Mhz
12	Giao tiếp với máy tính	USB-UART

Bảng 3.1: Thông số kỹ thuật của Arduino Nano

3.2.3. Khối đọc thẻ RFID

Khối đọc thẻ RFID có nhiệm vụ đọc mã định danh UID của thẻ xe, phục vụ quá trình xác thực phương tiện khi xe vào và xe ra khỏi bãi. Trong hệ thống, mã UID của thẻ RFID được sử dụng kết hợp với biển số xe nhận dạng từ camera để tạo thành hai yếu tố xác thực. Cách kết hợp này giúp hạn chế trường hợp sử dụng sai thẻ, nhầm xe hoặc phương tiện không hợp lệ ra khỏi bãi.

Hệ thống sử dụng hai module RFID RC522, trong đó một module được bố trí tại cổng vào và một module được bố trí tại cổng ra. Khi xe vào bãi, module RFID tại cổng vào đọc mã UID của thẻ và gửi dữ liệu về Arduino. Dữ liệu này sau đó được truyền lên khối xử lý trung tâm để lưu cùng với biển số xe và thời điểm vào bãi. Khi xe ra, module RFID tại cổng ra đọc lại mã UID của thẻ, hệ thống sẽ đối chiếu mã thẻ này với dữ liệu đã lưu trước đó để xác định phương tiện có hợp lệ hay không.

Module RFID RC522 được lựa chọn do có kích thước nhỏ gọn, chi phí thấp, dễ tích hợp với Arduino và hỗ trợ giao tiếp SPI. Module hoạt động ở tần số 13,56MHz, phù hợp với các loại thẻ Mifare thông dụng. Khoảng cách đọc thẻ ngắn giúp người dùng phải đưa thẻ lại gần đầu đọc, từ đó hạn chế việc đọc nhầm thẻ ở khoảng cách xa trong mô hình thực nghiệm.

Hai module RFID RC522 được kết nối với Arduino thông qua cùng một bus SPI. Các đường tín hiệu SCK, MOSI và MISO được dùng chung cho cả hai module, trong khi mỗi module sử dụng một chân chọn riêng để Arduino có thể phân biệt đầu đọc tại cổng vào và đầu đọc tại cổng ra. Cách kết nối này giúp tiết kiệm số lượng chân điều khiển trên Arduino, đồng thời vẫn bảo đảm khả năng đọc dữ liệu độc lập từ hai vị trí khác nhau trong hệ thống.



Hình 3.4: Module RFID RC522



Hình 3.5: Ký hiệu module RFID RC522

Thông số kỹ thuật của RFID RC522 :

STT	Thông số	Chi tiết
1	Tần số hoạt động	13,56MHz
2	Điện áp hoạt động	3,3VDC
3	Dòng tiêu thụ	Khoảng 13–26mA
4	Chuẩn giao tiếp	SPI
5	Tốc độ truyền dữ liệu tối đa	10Mbit/s
6	Khoảng cách đọc thẻ	Khoảng 0–60mm, tùy loại thẻ và điều kiện môi trường

Bảng 3.2: Thông số kỹ thuật của RFID RC522

Do module RFID RC522 hoạt động ở mức điện áp 3,3V, trong quá trình thiết kế cần chú ý đến mức điện áp giao tiếp giữa module và Arduino. Arduino Nano sử dụng mức logic 5V, vì vậy các đường tín hiệu từ Arduino sang module RFID cần được bảo đảm phù hợp với mức điện áp cho phép của RC522.

3.2.4. Khối động cơ

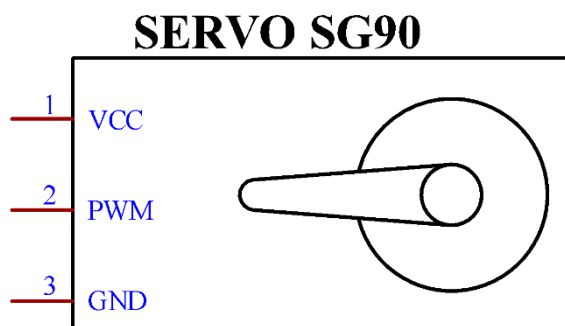
Khối động cơ có nhiệm vụ điều khiển cơ cấu đóng/mở thanh chắn tại cổng vào và cổng ra của bãi đỗ xe. Trong hệ thống, hai động cơ servo được sử dụng tương ứng với hai thanh chắn: một servo điều khiển thanh chắn cổng vào và một servo điều khiển thanh chắn cổng ra. Khi phương tiện được xác thực hợp lệ, khối điều khiển sẽ phát tín hiệu điều khiển để servo quay đến góc mở. Sau khi phương tiện đi qua, servo được điều khiển quay về góc đóng để kết thúc quá trình ra/vào.

Động cơ servo được lựa chọn vì có khả năng điều khiển góc quay theo vị trí xác định thông qua tín hiệu PWM, phù hợp với yêu cầu đóng/mở thanh chắn trong mô hình bãi đỗ xe. So với động cơ DC thông thường, servo không cần bổ sung thêm mạch phản hồi vị trí phức tạp, giúp quá trình điều khiển bằng Arduino đơn giản hơn. Ngoài ra, servo có kích thước nhỏ gọn, dễ lắp đặt và đáp ứng tốt yêu cầu của mô hình thực nghiệm.

Trong mô hình thực nghiệm, hệ thống sử dụng động cơ servo loại nhỏ để mô phỏng cơ cấu đóng/mở thanh chắn. Do thanh chắn trong mô hình có kích thước và khối lượng nhỏ, servo SG90 đáp ứng được yêu cầu điều khiển góc quay, dễ kết nối với Arduino và phù hợp với mục đích mô phỏng hoạt động của barrier trong bãi đỗ xe.



Hình 3.6: Động cơ micro servo SG90



Hình 3.7: Ký hiệu module servo SG90

Thông số kỹ thuật của servo SG90 :

STT	Thông số	Chi tiết
1	Điện áp hoạt động	4,8–6VDC
2	Mô-men xoắn	Khoảng 1,8kg·cm tại 4,8V
3	Tốc độ quay	Khoảng 0,1–0,12 giây/60°
4	Góc quay	Khoảng 0–180°
5	Khối lượng	Khoảng 9g
6	Kích thước	Khoảng 23 × 12,2 × 29mm
7	Tín hiệu điều khiển	PWM

Bảng 3.3: Thông số kỹ thuật của servo SG90

Khi thiết kế mạch điều khiển cho khối động cơ, cần chú ý đến vấn đề cấp nguồn cho servo. Do servo có dòng tiêu thụ thay đổi trong quá trình quay, đặc biệt tại thời điểm khởi động hoặc khi chịu tải, không nên cấp nguồn servo trực tiếp từ chân GPIO của Arduino. Servo nên được cấp nguồn từ khối nguồn riêng hoặc đường nguồn 5V có khả năng cung cấp dòng phù hợp. Tuy nhiên, mass của nguồn servo cần được nối chung với mass của Arduino để tín hiệu điều khiển PWM có cùng mức tham chiếu.

Ngoài ra, do quá trình hoạt động của servo có thể gây sụt áp hoặc nhiễu nguồn, mạch cấp nguồn cho khối động cơ cần được bố trí tụ lọc để ổn định điện áp. Trong trường hợp hệ thống sử dụng nhiều cơ cấu chấp hành hoạt động đồng thời, cần tính toán dòng cấp phù hợp để tránh ảnh hưởng đến hoạt động của vi điều khiển, cảm biến và các module khác trong hệ thống.

3.2.5. Khối đèn báo và còi cảnh báo

Khối đèn báo và còi cảnh báo có nhiệm vụ phản hồi trạng thái hoạt động của hệ thống tại mô hình bãi đỗ xe. Đây là khối đầu ra giúp người vận hành hoặc

người dùng có thể nhận biết trực tiếp tình trạng của bãi đỗ mà không cần chỉ quan sát trên giao diện phần mềm. Trong hệ thống, đèn LED được sử dụng để hiển thị trạng thái các ô đỗ, còn còi được sử dụng để phát tín hiệu cảnh báo trong các trường hợp bất thường.

Đèn báo được bố trí tương ứng với các ô đỗ trong mô hình. Mỗi ô đỗ sử dụng đèn LED để thể hiện trạng thái trống hoặc đã có xe. Khi cảm biến hồng ngoại tại ô đỗ phát hiện có phương tiện, Arduino sẽ cập nhật trạng thái và điều khiển LED tương ứng. Việc sử dụng đèn báo giúp trạng thái từng ô đỗ được thể hiện trực quan tại hiện trường, hỗ trợ người vận hành quan sát nhanh tình trạng của bãi đỗ.

Còi cảnh báo được sử dụng trong các tình huống cần thông báo cho người vận hành, chẳng hạn như khi bãi đỗ đã đầy, khi quá trình xác thực thẻ RFID và biển số xe không hợp lệ, hoặc khi cần cảnh báo có thao tác bất thường trong quá trình xe vào/ra. Khi xảy ra các tình huống này, Arduino sẽ kích hoạt còi để phát tín hiệu âm thanh, đồng thời hệ thống không cho phép mở thanh chắn nếu điều kiện xác thực không được đáp ứng.

Đèn LED và còi được lựa chọn do có cấu tạo đơn giản, dễ điều khiển bằng Arduino, chi phí thấp và phù hợp với mô hình thực nghiệm.

Về nguyên lý điều khiển, các LED báo trạng thái được kết nối với các chân Digital output của Arduino thông qua điện trở hạn dòng. Khi Arduino xuất mức logic phù hợp, LED sẽ sáng hoặc tắt tùy theo trạng thái của ô đỗ. Mỗi LED cần được mắc nối tiếp với điện trở hạn dòng để giới hạn dòng điện, tránh làm hỏng LED hoặc ảnh hưởng đến chân I/O của Arduino.

Đối với còi cảnh báo, không nên cấp nguồn trực tiếp từ chân GPIO nếu còi có dòng tiêu thụ lớn. Thay vào đó, còi cần được điều khiển thông qua mạch trung gian như transistor hoặc module relay. Arduino chỉ xuất tín hiệu điều khiển, còn dòng cấp cho còi được lấy từ khối nguồn phù hợp. Cách thiết kế này giúp bảo vệ vi điều khiển và bảo đảm còi hoạt động ổn định trong quá trình cảnh báo.

Khi thiết kế mạch, các thiết bị trong khối đèn báo và còi cần được nối chung mass với Arduino để bảo đảm tín hiệu điều khiển có cùng mức tham chiếu. Ngoài

ra, nếu sử dụng còi hoặc relay có dòng tiêu thụ lớn, cần kiểm tra khả năng cấp dòng của khối nguồn để tránh gây sụt áp làm ảnh hưởng đến hoạt động của các module khác trong hệ thống.

3.2.6. Khối camera

Khối camera có nhiệm vụ thu nhận hình ảnh phương tiện tại khu vực cổng vào và cổng ra, phục vụ quá trình xử lý ảnh và nhận dạng biển số xe. Trong hệ thống, camera đóng vai trò là thiết bị đầu vào hình ảnh cho khối xử lý trung tâm. Dữ liệu hình ảnh từ camera được sử dụng kết hợp với mã thẻ RFID để xác thực phương tiện trong quá trình xe vào và xe ra khỏi bãi.

Hệ thống sử dụng hai webcam USB, trong đó một webcam được bố trí tại cổng vào và một webcam được bố trí tại cổng ra. Việc sử dụng hai camera riêng biệt giúp tách biệt hai luồng xử lý xe vào và xe ra, đồng thời giúp khối xử lý trung tâm dễ dàng xác định nguồn ảnh tương ứng với từng cổng. Camera tại cổng vào dùng để ghi nhận biển số xe khi phương tiện đi vào bãi, còn camera tại cổng ra dùng để nhận dạng lại biển số khi phương tiện rời khỏi bãi.

Webcam USB được lựa chọn do có khả năng kết nối trực tiếp với máy tính mà không cần mạch giao tiếp trung gian. Thiết bị này tương thích tốt với các thư viện xử lý ảnh như OpenCV trong môi trường Python, dễ lắp đặt, chi phí thấp và phù hợp với mô hình thực nghiệm. Ngoài ra, việc sử dụng webcam giúp quá trình thu nhận hình ảnh được thực hiện trực tiếp trên khối xử lý trung tâm, giảm tải cho Arduino và bảo đảm các tác vụ xử lý ảnh được thực hiện trên thiết bị có năng lực tính toán phù hợp hơn.

Về vị trí lắp đặt, camera được đặt tại khu vực cổng vào và cổng ra, hướng về vị trí phương tiện dừng trước thanh chắn. Khi bố trí camera, cần bảo đảm biển số xe nằm trong vùng quan sát, không bị che khuất và có kích thước đủ lớn trong khung hình để phục vụ quá trình nhận dạng. Ngoài ra, góc đặt camera cần hạn chế độ nghiêng quá lớn của biển số, đồng thời khu vực chụp ảnh cần có điều kiện ánh sáng tương đối ổn định để nâng cao chất lượng ảnh đầu vào.

Thông số kỹ thuật của Webcam HD 720:

STT	Thông số	Chi tiết
1	Tỉ lệ khung hình	20fps / 30 fps
2	Độ phân giải chụp	2MP / 3.69MP / 8.29MP

Bảng 3.4: Thông số kỹ thuật của Webcam HD 720

Trong quá trình hoạt động, khi hệ thống phát hiện phương tiện tại cổng vào hoặc cổng ra, khối xử lý trung tâm sẽ thu nhận hình ảnh từ camera tương ứng. Hình ảnh sau đó được đưa vào chương trình xử lý ảnh để tiền xử lý, xác định vùng biển số và nhận dạng ký tự biển số bằng Tesseract OCR. Kết quả nhận dạng được sử dụng để lưu thông tin xe vào hoặc đối chiếu với dữ liệu đã lưu khi xe ra.

Chất lượng hình ảnh từ camera ảnh hưởng trực tiếp đến kết quả nhận dạng biển số. Vì vậy, trong mô hình thực nghiệm cần chú ý đến khoảng cách từ camera đến biển số, góc đặt camera, độ phân giải ảnh và điều kiện ánh sáng. Trường hợp hình ảnh bị mờ, biển số bị che khuất hoặc ánh sáng không ổn định có thể làm giảm độ chính xác của quá trình nhận dạng. Do đó, camera cần được cố định ở vị trí phù hợp để bảo đảm ảnh đầu vào có chất lượng đủ tốt cho quá trình xử lý.

3.2.7. Khối cảm biến hồng ngoại

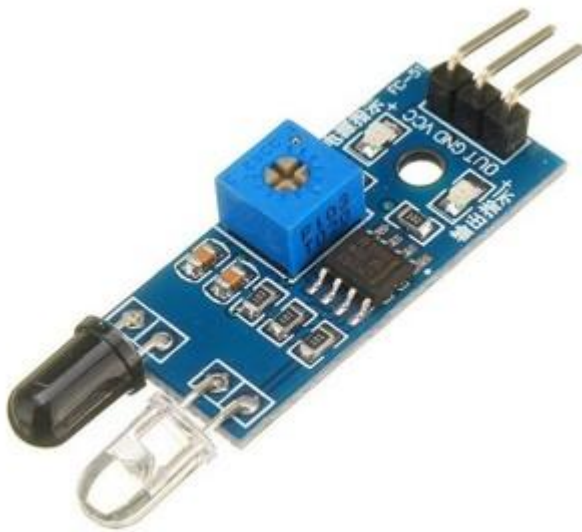
Khối cảm biến hồng ngoại có nhiệm vụ phát hiện sự xuất hiện của phương tiện tại các vị trí cần giám sát trong mô hình bãi đỗ xe. Đây là một trong các khối đầu vào quan trọng của hệ thống, cung cấp dữ liệu cho khối điều khiển để phục vụ quá trình xử lý xe vào, xe ra và cập nhật trạng thái các ô đỗ.

Trong hệ thống, cảm biến hồng ngoại được bố trí tại hai khu vực chính. Thứ nhất, cảm biến được đặt tại cổng vào và cổng ra để phát hiện phương tiện đang dừng trước thanh chắn. Tín hiệu này được sử dụng làm điều kiện kích hoạt quá trình quét thẻ RFID, thu nhận hình ảnh biển số và xử lý logic đóng/mở thanh chắn. Thứ hai, cảm biến được đặt tại từng ô đỗ để xác định trạng thái trống hoặc đã có

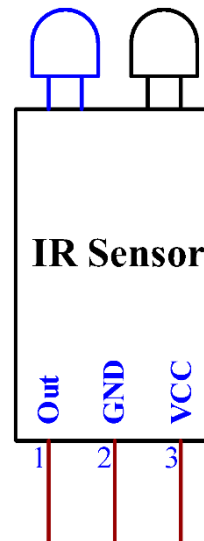
xe của ô đồ. Dữ liệu trạng thái các ô đồ được gửi về khối xử lý trung tâm để cập nhật lên giao diện giám sát và lưu vào cơ sở dữ liệu.

Cảm biến hồng ngoại được lựa chọn do có cấu tạo đơn giản, giá thành thấp, kích thước nhỏ gọn và dễ tích hợp với Arduino. Module cảm biến có ngõ ra dạng số, vì vậy Arduino có thể đọc trực tiếp trạng thái phát hiện vật cản thông qua các chân Digital hoặc các chân Analog được sử dụng như ngõ vào số. Ngoài ra, cảm biến có tích hợp biến trở cho phép điều chỉnh khoảng cách phát hiện, phù hợp với yêu cầu lắp đặt trong mô hình thực nghiệm.

Hệ thống sử dụng tổng cộng 5 cảm biến hồng ngoại, bao gồm 2 cảm biến đặt tại cổng vào và cổng ra, cùng 3 cảm biến đặt tại 3 ô đồ xe. Các cảm biến tại ô đồ được dùng để xác định sự thay đổi trạng thái của từng vị trí đồ. Khi có phương tiện đi vào ô đồ, cảm biến phát hiện vật cản và gửi tín hiệu về Arduino. Ngược lại, khi xe rời khỏi ô đồ, tín hiệu cảm biến thay đổi, từ đó hệ thống cập nhật trạng thái ô đồ từ đã có xe sang trống.



Hình 3.8: Cảm biến hồng ngoại IR01



Hình 3.9: Ký hiệu cảm biến hồng ngoại

Thông số kỹ thuật của cảm biến hồng ngoại:

STT	Thông số	Chi tiết
1	Điện áp hoạt động	3,3–5VDC
2	Ngõ ra tín hiệu	Digital TTL
3	Dòng điện tiêu thụ	> 20 mA
4	Nhiệt độ làm việc	-10°C ~ +50°C
5	Kích thước vật lý	3.2 cm x 1.4 cm (thiết kế tối giản, tiết kiệm không gian lắp đặt)

Bảng 3.5: Thông số kỹ thuật của cảm biến hồng ngoại

Trong mô hình thực nghiệm, các cảm biến tại ô đỗ được kết nối với Arduino và được đọc theo chu kỳ để cập nhật trạng thái bãi đỗ. Dữ liệu từ các cảm biến này được đóng gói cùng các thông tin trạng thái khác và truyền lên khối xử lý trung tâm thông qua giao tiếp UART. Khi phát hiện trạng thái ô đỗ thay đổi so với lần đọc trước, hệ thống sẽ cập nhật dữ liệu vào cơ sở dữ liệu và hiển thị trạng thái mới trên dashboard website.

Khi thiết kế và lắp đặt khối cảm biến hồng ngoại, cần chú ý điều chỉnh khoảng cách phát hiện phù hợp với vị trí đặt xe trong mô hình. Nếu khoảng cách phát hiện quá ngắn, cảm biến có thể không nhận biết được phương tiện; ngược lại, nếu khoảng cách quá xa, cảm biến có thể nhận nhầm vật thể ngoài vùng cần giám sát. Ngoài ra, cảm biến hồng ngoại có thể bị ảnh hưởng bởi góc phản xạ, màu sắc bề mặt vật thể và điều kiện ánh sáng môi trường, do đó cần cố định cảm biến ở vị trí ổn định để bảo đảm kết quả phát hiện đáng tin cậy.

3.2.8. Khối nguồn

Khối nguồn có nhiệm vụ cung cấp điện áp và dòng điện phù hợp cho các thành phần phần cứng trong hệ thống, bảo đảm các khối như Arduino, cảm biến hồng ngoại, module RFID, động cơ servo, đèn báo và còi cảnh báo hoạt động ổn định. Đây là khối quan trọng trong thiết kế phần cứng, vì sự không ổn định của

nguồn cấp có thể làm ảnh hưởng đến quá trình đọc cảm biến, điều khiển thanh chắn hoặc truyền dữ liệu giữa Arduino và máy tính.

Trong hệ thống, nguồn cấp được phân chia theo từng nhóm thiết bị. Máy tính cục bộ sử dụng nguồn riêng của máy tính hoặc laptop để vận hành khối xử lý trung tâm, camera USB, cơ sở dữ liệu và giao diện giám sát. Arduino Nano có thể được cấp nguồn thông qua cổng USB kết nối với máy tính, đồng thời cũng sử dụng kết nối này để truyền dữ liệu UART. Các thiết bị ngoại vi như cảm biến, RFID, servo, đèn báo và còi cảnh báo được cấp nguồn từ khối nguồn của mô hình, tùy theo điện áp làm việc của từng thiết bị.

STT	Thành phần	Điện áp sử dụng	Dòng tiêu thụ ước tính
1	Arduino Nano	5VDC	Khoảng 50–100mA
2	Module RFID RC522	3,3VDC/module	Khoảng 13–26mA/module
3	Cảm biến hồng ngoại	3,3–5VDC/module	Khoảng 10–20mA/module
4	Động cơ servo SG90	4,8–6VDC/module	Thay đổi theo tải
5	LED báo trạng thái	5VDC/led	Khoảng 5–20mA/LED
6	Còi cảnh báo	5VDC	< 25mA
7	Camera	5VDC	≤ 500mA

Bảng 3.6: Điện áp và dòng tiêu thụ ước tính của các thành phần trong hệ thống

Từ bảng ước lượng trên, có thể thấy các tải như servo và còi cảnh báo là những thành phần cần được chú ý khi thiết kế nguồn, do dòng tiêu thụ có thể tăng tại thời điểm hoạt động. Vì vậy, nguồn cấp cho các thiết bị ngoại vi cần có khả

năng cung cấp dòng lớn hơn tổng dòng tiêu thụ ước tính và có dự phòng cho các thời điểm tải tăng đột ngột.

Trong trường hợp sử dụng adapter ngoài, điện áp đầu vào có thể được hạ xuống các mức điện áp phù hợp thông qua mạch ổn áp hoặc module giảm áp. Đối với các tải nhỏ như cảm biến, LED hoặc mạch logic, có thể sử dụng IC ổn áp tuyến tính nếu dòng tiêu thụ không lớn. Tuy nhiên, đối với các tải có dòng cao hơn như servo hoặc còi, nên sử dụng nguồn 5V riêng hoặc module giảm áp DC-DC để đảm bảo hiệu suất và hạn chế hiện tượng nóng linh kiện.

Ngoài ra, do servo và còi có thể gây sụt áp hoặc nhiễu nguồn khi hoạt động, mạch nguồn cần được bố trí thêm tụ lọc tại các nhánh cấp nguồn cho tải. Tụ lọc có tác dụng giảm dao động điện áp tức thời, giúp hạn chế ảnh hưởng của các tải động đến Arduino và các module cảm biến. Với các tải cảm ứng hoặc relay, có thể bổ sung diode bảo vệ nhằm giảm xung điện áp ngược trong quá trình đóng/ngắt.

3.3. THIẾT KẾ PHẦN MỀM

Phần mềm của hệ thống được thiết kế theo kiến trúc phân tầng, phù hợp với cấu trúc phần cứng đã trình bày ở mục 3.2. Trong đó, tầng điều khiển thiết bị ngoại vi được triển khai trên Arduino, còn tầng xử lý trung tâm được triển khai trên máy tính cục bộ. Cách phân chia này giúp tách biệt giữa các tác vụ điều khiển mức thấp tại hiện trường và các tác vụ xử lý dữ liệu có độ phức tạp cao hơn.

Tầng phần mềm trên Arduino chịu trách nhiệm đọc tín hiệu từ cảm biến hồng ngoại, đọc mã thẻ RFID, điều khiển động cơ servo, đèn báo và còi cảnh báo. Ngoài ra, Arduino còn thực hiện đóng gói dữ liệu trạng thái của hệ thống và truyền lên máy tính thông qua giao tiếp UART. Các tác vụ trên Arduino được thiết kế đơn giản, có tính lặp lại cao và phù hợp với yêu cầu điều khiển thời gian gần thực của mô hình.

Tầng phần mềm trên máy tính cục bộ đảm nhận các chức năng xử lý chính của hệ thống, bao gồm tiếp nhận dữ liệu từ Arduino, thu nhận hình ảnh từ camera, nhận dạng biển số xe, xử lý logic xe vào/ra, tính phí gửi xe, lưu trữ dữ liệu và cung

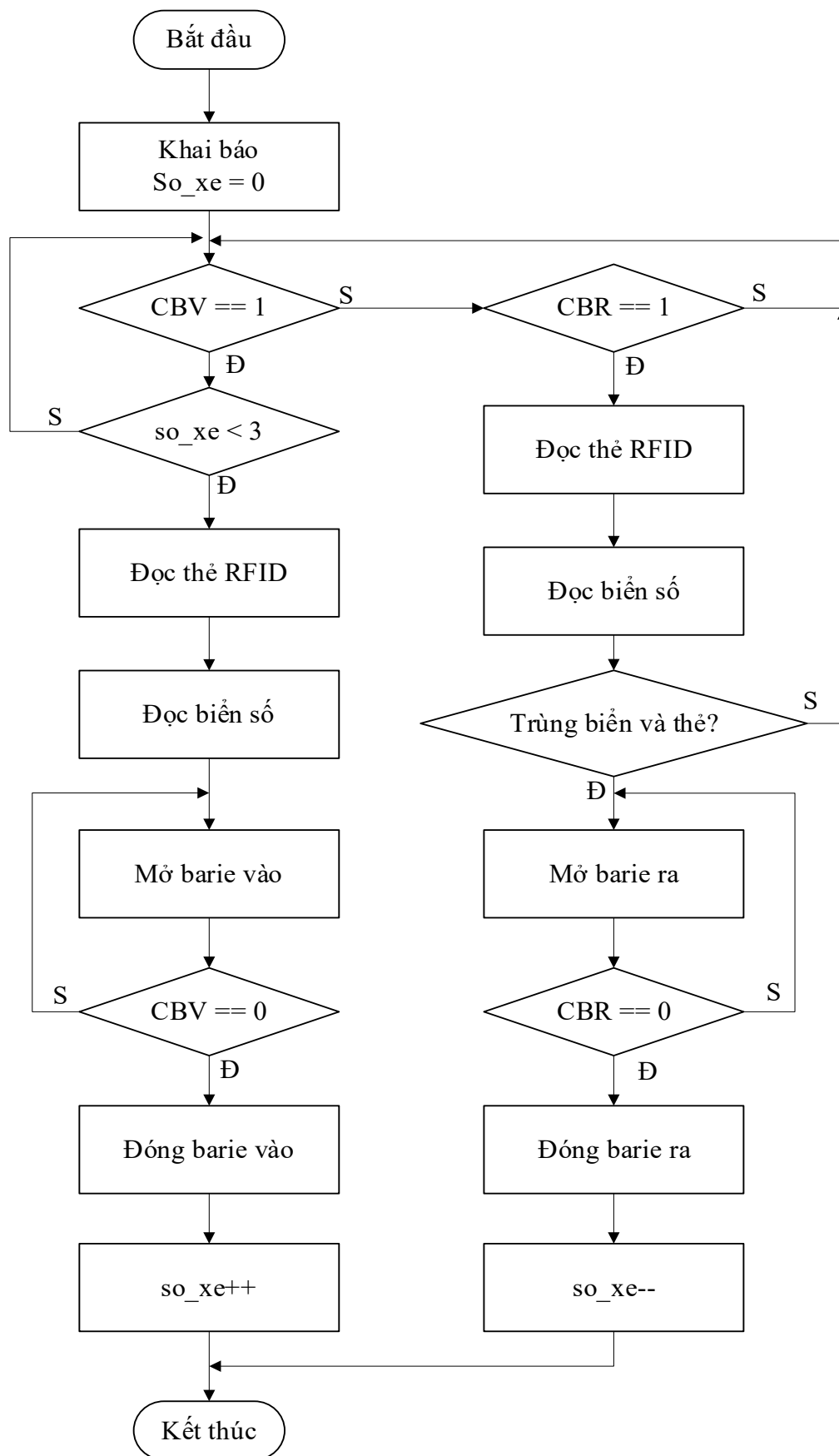
cấp giao diện giám sát. Các thành phần phần mềm trên máy tính được xây dựng bằng Python, kết hợp với các thư viện và công cụ như OpenCV, Tesseract OCR, SQLite, PyQt5 và Flask.

Bên cạnh các chức năng điều khiển và xử lý nghiệp vụ, hệ thống còn cung cấp dashboard website để người vận hành theo dõi trạng thái bãi đỗ, lịch sử xe vào/ra, các biểu đồ thống kê và kết quả dự báo số chỗ trống. Dữ liệu trên dashboard được cập nhật từ khối xử lý trung tâm, giúp quá trình giám sát được thực hiện trực quan và gần thời gian thực.

Nội dung phần này trình bày thiết kế phần mềm của hệ thống theo từng thành phần chính, bao gồm chương trình điều khiển trên Arduino, chương trình xử lý trung tâm trên máy tính, module nhận dạng biển số xe, cơ sở dữ liệu, dashboard website và mô hình dự báo chỗ trống.

3.3.1. Lưu đồ chương trình Arduino

Chương trình trên Arduino vận hành theo một vòng lặp chính: khởi tạo các chân tín hiệu, module RFID, servo và cổng serial; sau đó liên tục đọc trạng thái cảm biến và đầu đọc RFID, xác định chế độ vận hành (Auto/Manual), thực hiện logic điều khiển thanh chắn theo luồng xe vào và xe ra đã mô tả ở mục 3.1.3, cập nhật đèn LED báo ô đỗ, rồi đóng gói toàn bộ trạng thái thành chuỗi dữ liệu và gửi lên máy tính.

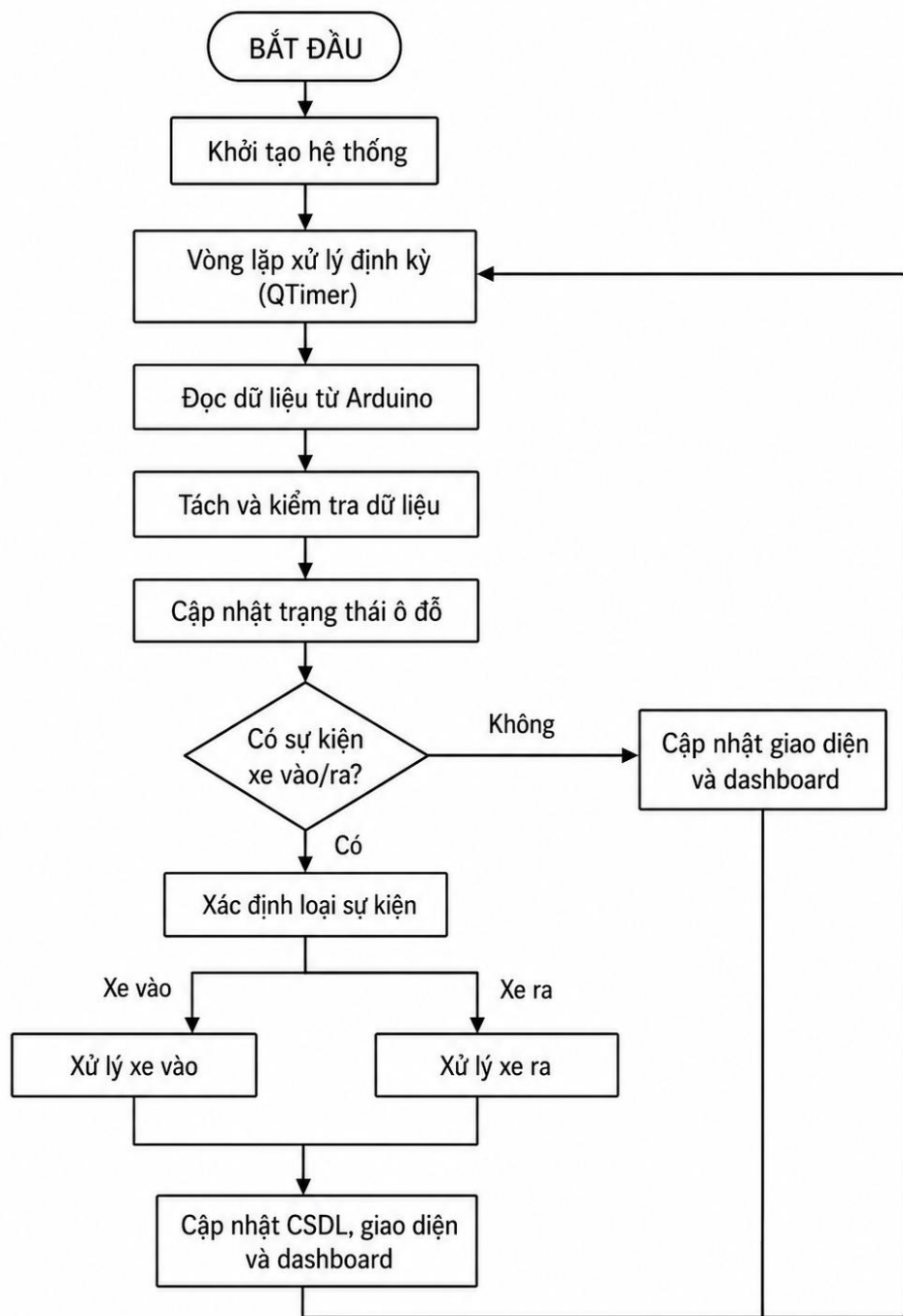


Hình 3.10: Lưu đồ chương trình Arduino

Việc gửi dữ liệu lên máy tính được thực hiện ở hai thời điểm để bảo đảm thông tin luôn được cập nhật: gửi định kỳ mỗi 200ms trong vòng lặp chính (hàm `guidulieu()`), và gửi bổ sung ngay khi xảy ra sự kiện liên quan RFID xe vào hoặc xe ra (hàm `guidulieu2()`). Hai hàm này được bổ sung 3 dòng lệnh giống nhau để đọc và gửi trạng thái 3 ô đỗ (`o1`, `o2`, `o3`), nối vào cuối chuỗi dữ liệu gốc mà không làm thay đổi thứ tự 7 trường ban đầu, qua đó không ảnh hưởng đến phần xử lý đã có ở máy tính.

3.3.2. Lưu đồ chương trình ứng dụng trên máy tính

Ứng dụng trên máy tính được xây dựng trên nền giao diện PyQt5, trong đó một bộ định thời (QTimer) gọi hàm xử lý chính theo chu kỳ 50ms (tương đương 20 lần mỗi giây) để đọc dữ liệu mới từ cổng serial và cập nhật toàn bộ trạng thái nghiệp vụ.



Hình 3.11: Lưu đồ vòng lặp xử lý chính trên máy tính

Một vấn đề kỹ thuật cần giải quyết là: các cờ trạng thái nhận được từ Arduino (ví dụ cờ báo có sự kiện xe vào, cờ báo xe ra hợp lệ) giữ nguyên giá trị 1 trong nhiều vòng lặp liên tiếp không chỉ đúng một vòng do sự kiện vật lý (xe đi qua cảm biến) kéo dài hơn 50ms. Nếu ghi cơ sở dữ liệu mỗi khi đọc thấy cờ bằng 1, một sự kiện duy nhất có thể bị ghi lặp lại hàng chục lần. Giải pháp được áp dụng

là kỹ thuật dò biên (edge detection): chỉ ghi dữ liệu tại đúng thời điểm cờ chuyển từ 0 sang 1 lần đầu tiên, đánh dấu bằng một biến cờ nội bộ, và chỉ mở khóa cờ này khi tín hiệu từ Arduino trở lại 0. Ba luồng ghi dữ liệu áp dụng nguyên lý này được trình bày dưới đây.

3.3.2.1. Luồng cập nhật trạng thái ô đỗ

Sau mỗi lần đọc dữ liệu serial, ứng dụng so sánh bộ ba giá trị (o1, o2, o3) vừa nhận với giá trị của lần đọc trước đó (lưu trong biến `_prev_slots`, khởi tạo bằng `[-1, -1, -1]` để đảm bảo lần đọc đầu tiên luôn được ghi nhận). Nếu phát hiện khác biệt, hệ thống cập nhật bảng slots cho cả 3 ô và ghi thêm một dòng lịch sử vào bảng `slot_history`; nếu giống nhau, không thực hiện ghi nào, tránh việc bảng lịch sử bị làm đầy bởi các lần đọc lặp lại cùng một trạng thái.

3.3.2.2. Luồng ghi sự kiện xe vào

Điều kiện ghi nhận một sự kiện xe vào là: cờ báo có thẻ quét tại cổng vào bằng 1, cờ báo xác thực hợp lệ bằng 1, và số xe hiện tại trong bãi nhỏ hơn 3. Khi điều kiện này lần đầu được thỏa mãn (cờ nội bộ đánh dấu chưa ghi), hệ thống lấy biển số đã nhận diện được từ camera cổng vào và ghi một sự kiện loại "in" vào bảng events kèm thời điểm xảy ra, sau đó đặt cờ nội bộ thành đã ghi để khóa lại. Cờ chỉ được mở khóa trở lại khi tín hiệu từ Arduino báo sự kiện đã kết thúc (trở về 0), sẵn sàng cho lượt xe vào tiếp theo.

3.3.2.3. Luồng ghi sự kiện xe ra

Tương tự luồng xe vào, khi cờ báo xe ra hợp lệ (đã đối chiếu khớp cả thẻ RFID và biển số) lần đầu chuyển sang 1, hệ thống tính thời lượng gửi xe và mức phí tương ứng, ghi một sự kiện loại "out" vào bảng events kèm các thông tin này, rồi khóa cờ nội bộ lại để tránh ghi lặp; cờ được mở khóa khi tín hiệu từ Arduino trở về 0.

Việc áp dụng cùng một nguyên lý dò biên cho cả ba luồng giúp đảm bảo mỗi sự kiện vật lý chỉ tương ứng với đúng một bản ghi trong cơ sở dữ liệu, bất kể

vòng lặp xử lý chạy với tần suất cao hơn nhiều so với tốc độ diễn ra của sự kiện thực tế.

Khởi động máy chủ web song song với giao diện:

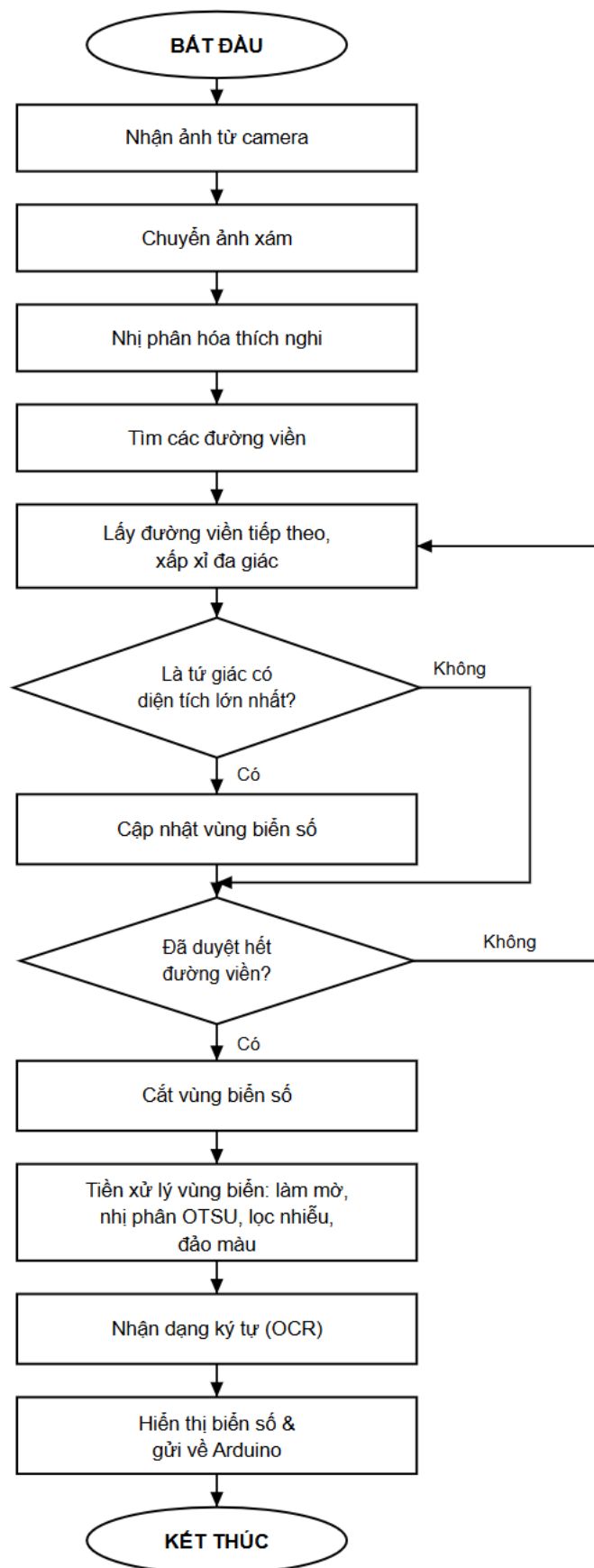
Hai dòng lệnh khởi tạo cơ sở dữ liệu và khởi động luồng (thread) chạy máy chủ Flask được đặt trước vòng lặp tạo giao diện chính của ứng dụng, vì vòng lặp này có thể khởi tạo lại đối tượng giao diện nhiều lần trong quá trình chạy nếu khởi động Flask bên trong vòng lặp đó sẽ phát sinh lỗi xung đột cổng mạng do máy chủ bị khởi động lại trong khi cổng cũ vẫn còn đang được giữ. Luồng Flask được đánh dấu là luồng nền (daemon), giúp nó tự động kết thúc khi ứng dụng chính tắt mà không cần xử lý dọn dẹp riêng. Sau mỗi lần đọc serial, ứng dụng cũng đồng thời cập nhật một cấu trúc dữ liệu dùng chung để Flask đọc và trả về cho giao diện web các thông tin theo thời gian thực (chế độ vận hành, trạng thái hai thanh chắn, biên số vừa nhận diện) mà không cần truy vấn cơ sở dữ liệu cho những dữ liệu có tính tức thời này.

3.3.3. Module nhận diện biên số xe

Module nhận diện biên số được xây dựng theo phương pháp xử lý ảnh cổ điển, không sử dụng mạng nơ-ron học sâu, phù hợp với yêu cầu xử lý thời gian thực trên cấu hình máy tính phổ thông và quy mô của mô hình thực nghiệm. Quy trình xử lý gồm các bước tuần tự sau:

1. Thu nhận khung hình từ camera tương ứng (cổng vào hoặc cổng ra) tại thời điểm phương tiện được phát hiện.
2. Chuyển ảnh màu sang ảnh xám (grayscale) để giảm khối lượng tính toán cho các bước xử lý tiếp theo.
3. Phân ngưỡng ảnh (thresholding/binarization) để tách vùng sáng/tối, làm nổi bật các chi tiết có khả năng là ký tự và viền biên số.
4. Dò biên (contour detection) trên ảnh đã phân ngưỡng để xác định các vùng có hình dạng và tỉ lệ khung hình đặc trưng của biên số xe, từ đó khoanh vùng ứng viên chứa biên số.

5. Cắt (crop) vùng ảnh ứng viên chứa biển số ra khỏi khung hình gốc để xử lý riêng.
6. Đưa vùng ảnh đã cắt vào công cụ Tesseract OCR để nhận dạng ký tự, trả về chuỗi văn bản biển số xe.



Hình 3.12: Lưu đồ module nhận diện biển số xe

Kết quả biển số nhận diện được tại cổng vào được lưu cùng mã thẻ RFID đọc được tại cùng thời điểm, tạo thành một cặp dữ liệu xác thực. Khi xe ra, quy trình nhận diện được lặp lại với camera cổng ra; chuỗi biển số và mã thẻ RFID mới được đối chiếu với cặp dữ liệu đã lưu để quyết định cho phép xe ra hay yêu cầu xác thực lại, như đã mô tả tại mục 3.1.3.

3.3.4. Thiết kế cơ sở dữ liệu

Hệ thống sử dụng SQLite một cơ sở dữ liệu nhúng lưu toàn bộ dữ liệu trong một tệp duy nhất, không yêu cầu cài đặt hay vận hành một máy chủ cơ sở dữ liệu riêng biệt, phù hợp với quy mô triển khai cục bộ trên một máy tính. Lược đồ dữ liệu gồm ba bảng, có vai trò khác nhau và được trình bày trong Bảng 3.7.

Bảng	Các trường chính	Vai trò
slots	id, state, updated_at	Trạng thái hiện tại của 3 ô đỗ; luôn cố định 3 dòng, bị ghi đè mỗi khi trạng thái đổi; nguồn dữ liệu cho bản đồ bãi xe trên dashboard
slot_history	id, slot_id, state, so_xe, changed_at	Lịch sử mỗi lần thay đổi trạng thái ô đỗ; tích lũy theo thời gian; là nguồn dữ liệu duy nhất cho mô hình dự đoán
events	id, type, plate, occurred_at, duration_s, fee	Lịch sử từng lượt xe vào (type="in") hoặc ra (type="out"), kèm biển số, thời lượng gửi và phí (chỉ có khi ra)

Bảng 3.7: Lược đồ ba bảng dữ liệu trong parking.db

Quy tắc ghi dữ liệu vào bảng slot_history chỉ thực hiện khi trạng thái một ô thực sự thay đổi so với lần đọc trước (xem luồng dò biên tại mục 3.3.2.1), tránh tích lũy dữ liệu trùng lặp không mang thông tin mới.

Truy cập đồng thời từ nhiều luồng:

Vì ứng dụng PyQt5 (luồng chính, đảm nhận việc ghi dữ liệu) và máy chủ Flask (luồng phụ, chỉ đọc dữ liệu để trả về cho giao diện web) cùng truy cập một tệp cơ sở dữ liệu, kết nối SQLite được mở với tùy chọn cho phép sử dụng từ nhiều luồng (check_same_thread=False). SQLite hỗ trợ tốt nhiều luồng đọc đồng thời trong khi chỉ có một luồng ghi, nên mô hình một luồng ghi một luồng đọc của hệ thống không phát sinh tranh chấp khóa đáng kể ở quy mô dữ liệu của mô hình thực nghiệm.

3.3.5. Thiết kế web dashboard

Bên cạnh giao diện vận hành PyQt5 cài đặt trực tiếp trên máy tính điều khiển, hệ thống cung cấp thêm một giao diện web (dashboard) chạy trên máy chủ Flask, cho phép giám sát trạng thái và thống kê của bãi đỗ từ một trình duyệt bất kỳ trong cùng mạng cục bộ.

Mô hình chạy song song với giao diện PyQt5:

Máy chủ Flask được khởi động trong một luồng riêng, độc lập với luồng giao diện chính, theo cơ chế đã trình bày tại mục 3.3.2. Toàn bộ trang dashboard (HTML, CSS và mã JavaScript) được gói trong một tệp template duy nhất, được Flask trả về khi có yêu cầu tới địa chỉ gốc.

Giao diện cung cấp 4 điểm cuối (API) phục vụ việc lấy dữ liệu, được tóm tắt trong Bảng 3.8.

Endpoint	Phương thức	Dữ liệu trả về
/	GET	Trang web dashboard (HTML)

Endpoint	Phương thức	Dữ liệu trả về
/api/status	GET	Trạng thái thời gian thực: 3 ô đỗ, số chỗ trống, chế độ vận hành, trạng thái 2 thanh chắn, biển số vừa nhận diện
/api/history	GET	Danh sách sự kiện ra/vào gần nhất (mặc định 50, có thể tùy chỉnh qua tham số limit)
/api/stats	GET	Thống kê lưu lượng xe và doanh thu theo giờ/ngày, phục vụ vẽ biểu đồ
/api/predict	GET	Kết quả dự đoán số chỗ trống cho 24 giờ tiếp theo

Bảng 3.8: Các API do máy chủ Flask cung cấp

Cơ chế tự cập nhật giao diện (polling):

Giao diện web không sử dụng WebSocket mà áp dụng cơ chế truy vấn định kỳ (polling) bằng JavaScript: định kỳ gọi từng API và cập nhật lại nội dung trang. Tần suất gọi được tách biệt theo loại dữ liệu, vì dữ liệu trạng thái thay đổi nhanh hơn nhiều so với dữ liệu thống kê và dự đoán, như trình bày trong Bảng 3.9.

API	Tần suất	Phần giao diện được cập nhật
/api/status	1,5 giây	Đồng hồ, huy hiệu chế độ, đèn trạng thái 2 thanh chắn, bản đồ 3 ô đỗ, số chỗ trống, sự kiện gần nhất
/api/history	5 giây	Bảng lịch sử 50 lượt ra/vào gần nhất
/api/stats	30 giây	3 biểu đồ thống kê (lượt xe theo ngày, giờ cao điểm, doanh thu theo ngày)
/api/predict	60 giây	Biểu đồ dự đoán chỗ trống 24 giờ tới

Bảng 3.9: Tần suất truy vấn (polling) của giao diện web

Cấu trúc nội dung trang dashboard:

Trang dashboard được tổ chức thành 7 phần, liệt kê trong Bảng 3.10.

Phần	Nội dung
A. Header	Đồng hồ thời gian thực, huy hiệu chế độ Auto/Manual, đèn trạng thái 2 thanh chắn
B. Tổng quan	Số chỗ trống hiển thị nổi bật, thanh tiến độ tỉ lệ phần trăm lấp đầy
C. Bản đồ bãi xe	3 thẻ tương ứng 3 ô đỗ; màu xanh = trống, màu đỏ = có xe; hiệu ứng nhấp nháy khi đổi trạng thái
D. Sự kiện gần nhất	Biển số, loại sự kiện (vào/ra), thời gian, thời lượng gửi, phí
E. Lịch sử	Bảng 50 lượt ra/vào gần nhất, tự làm mới định kỳ
F. Biểu đồ thống kê	3 biểu đồ: lượt xe theo ngày, giờ cao điểm, doanh thu theo ngày
G. Dự đoán	Biểu đồ chỗ trống dự kiến trong 24 giờ tới

Bảng 3.10: Cấu trúc 7 phần của trang web dashboard

Ba biểu đồ tại phần F được vẽ bằng thư viện Chart.js (phiên bản 4.4.0, tải từ CDN): biểu đồ cột nhóm thể hiện số lượt vào/ra theo từng ngày; biểu đồ cột thể hiện tổng số lượt ra/vào theo từng giờ trong ngày (0–23h, gộp tất cả các ngày) để xác định khung giờ cao điểm; và biểu đồ đường có tô màu thể hiện doanh thu theo từng ngày.



Hình 3.13: Giao diện web dashboard

Một số lưu ý kỹ thuật trong cài đặt:

- Giao diện sử dụng tông màu tối (dark mode) với bảng màu cố định, gồm các biến CSS quy định màu nền trang, màu nền thẻ, màu ô trống, màu ô có xe và màu nhấn.
- Khi chạy Flask ở chế độ production (debug=False, bắt buộc để vận hành ổn định), bộ máy template mặc định lưu bộ nhớ đệm bản dựng HTML sau

lần hiển thị đầu tiên, khiến các thay đổi sau đó trên tệp template không được phản ánh; vấn đề này được khắc phục bằng cách bật cấu hình tự động tải lại template từ đĩa mỗi lần có yêu cầu.

3.3.6. Mô hình dự đoán chỗ trống

Hệ thống cung cấp một mô hình dự đoán số chỗ trống dự kiến cho 24 giờ kế tiếp, dựa trên giả định bãi đỗ xe có tính chu kỳ theo thời gian: lưu lượng xe trong một giờ nhất định của một ngày trong tuần có xu hướng lặp lại tương tự ở các tuần khác cùng giờ, cùng ngày trong tuần. Đây là mô hình nền thống kê phi tham số (seasonal-mean / climatology baseline), không phải mô hình học máy hay học sâu, được lựa chọn làm mốc so sánh đầu tiên do dữ liệu lịch sử hiện còn hạn chế.

3.3.6.1. Xây dựng bảng tra cứu từ dữ liệu lịch sử

Toàn bộ dữ liệu trong bảng slot_history được nhóm theo cặp (thứ trong tuần, giờ trong ngày), từ đó tính trung bình số chỗ trống quan sát được tại mỗi nhóm, bằng một truy vấn SQL nhóm dữ liệu (GROUP BY) trên hai trường thời gian trích xuất từ cột changed_at. Kết quả là một bảng tra cứu ánh xạ mỗi cặp (thứ, giờ) sang giá trị trung bình chỗ trống và số lượng mẫu quan sát được, ví dụ minh họa tại Bảng 3.11.

Thứ	Giờ	Trung bình chỗ trống	Số mẫu
Thứ Sáu	8h	0,8	12
Thứ Bảy	10h	2,1	8
Chủ Nhật	14h	2,5	5

Bảng 3.11: Ví dụ một số dòng trong bảng tra cứu trung bình theo (thứ, giờ)

3.3.6.2. Dự đoán cho 24 giờ tiếp theo

Với mỗi giờ trong 24 giờ kế tiếp tính từ thời điểm hiện tại, hệ thống xác định giờ đó rơi vào thứ mấy trong tuần, tra cứu giá trị trung bình tương ứng trong

bảng đã xây dựng ở bước trên, và trả về dưới dạng danh sách 24 dự đoán kèm nhãn thời gian. Nếu một cặp (thứ, giờ) chưa có dữ liệu lịch sử, hệ thống trả về giá trị rỗng cho khung giờ đó thay vì suy diễn không có cơ sở, và biểu đồ sẽ hiển thị khoảng trống tương ứng.

Vì Python và SQLite quy ước đánh số thứ trong tuần khác nhau (Python coi thứ Hai là ngày 0, SQLite coi Chủ Nhật là ngày 0), hệ thống áp dụng công thức chuyển đổi: thứ theo SQLite = (thứ theo Python + 1) chia lấy dư 7, để đảm bảo tra cứu đúng nhóm dữ liệu.

3.3.6.3. Đánh giá độ chính xác của mô hình

Để đánh giá khách quan chất lượng dự đoán, dữ liệu lịch sử được chia theo trình tự thời gian không chia ngẫu nhiên thành 80% dữ liệu cũ hơn dùng để xây dựng mô hình (huấn luyện) và 20% dữ liệu mới hơn dùng để kiểm tra. Cách chia theo thời gian này tránh hiện tượng rò rỉ dữ liệu (data leakage) có thể xảy ra nếu chia ngẫu nhiên, vì khi đó mô hình có thể "nhìn thấy" những thời điểm tương lai gần với thời điểm cần dự đoán trong tập kiểm tra.

Trên tập kiểm tra, sai số dự đoán được đo bằng hai chỉ số phổ biến trong bài toán dự báo số lượng: MAE (Mean Absolute Error sai số tuyệt đối trung bình) và RMSE (Root Mean Square Error căn bậc hai trung bình bình phương sai số, có tính chất phạt nặng hơn các sai số lớn). Việc đánh giá chỉ được thực hiện khi bảng slot_history có ít nhất 30 bản ghi; nếu chưa đủ, hệ thống tạm hoãn đánh giá cho đến khi tích lũy thêm dữ liệu.

Mô hình seasonal-mean được xác định rõ là mô hình nền (baseline) mức chuẩn tối thiểu cần đạt được nhằm làm mốc so sánh khi hệ thống được nâng cấp lên các mô hình dự đoán phức tạp hơn trong tương lai.

CHƯƠNG 4

KẾT QUẢ

4.1. KẾT QUẢ THỰC HIỆN MÔ HÌNH

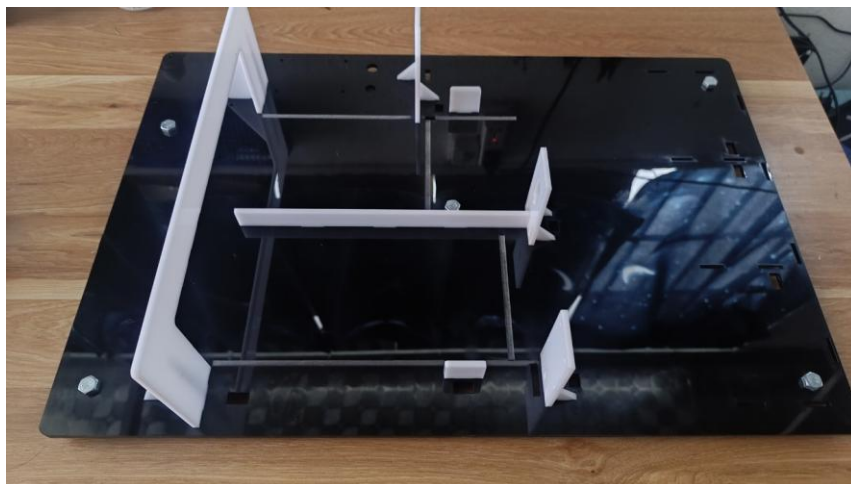
Mô hình mô phỏng của đề tài đã được hoàn thiện nhằm kiểm chứng các chức năng chính của hệ thống, bao gồm nhận diện biển số, đọc thẻ RFID, điều khiển barrier, giám sát trạng thái 3 ô đỗ và hiển thị dữ liệu trên dashboard.

Vật liệu làm khung cho mô hình là mica có độ dày 5mm và 3mm



Hình 4.1: Mica

Lắp đặt phân công vào ra



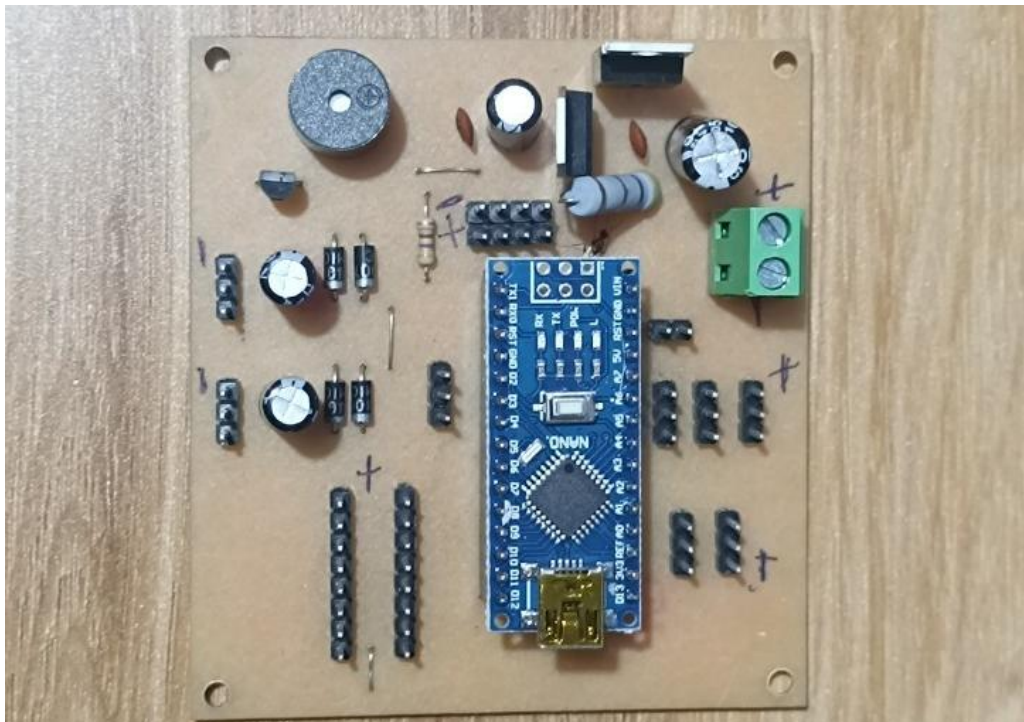
Hình 4.2: Lắp đặt cổng vào ra

Lắp đặt phần bãi đỗ



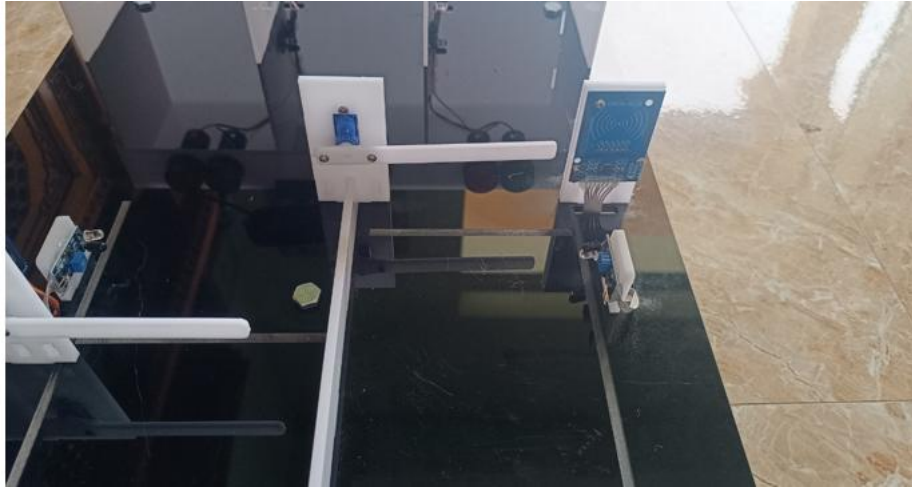
Hình 4.3: Lắp đặt bãi đỗ

Gia công mạch điện điều khiển



Hình 4.4: Gia công mạch điện điều khiển

Lắp đặt các cảm biến, module đọc thẻ từ



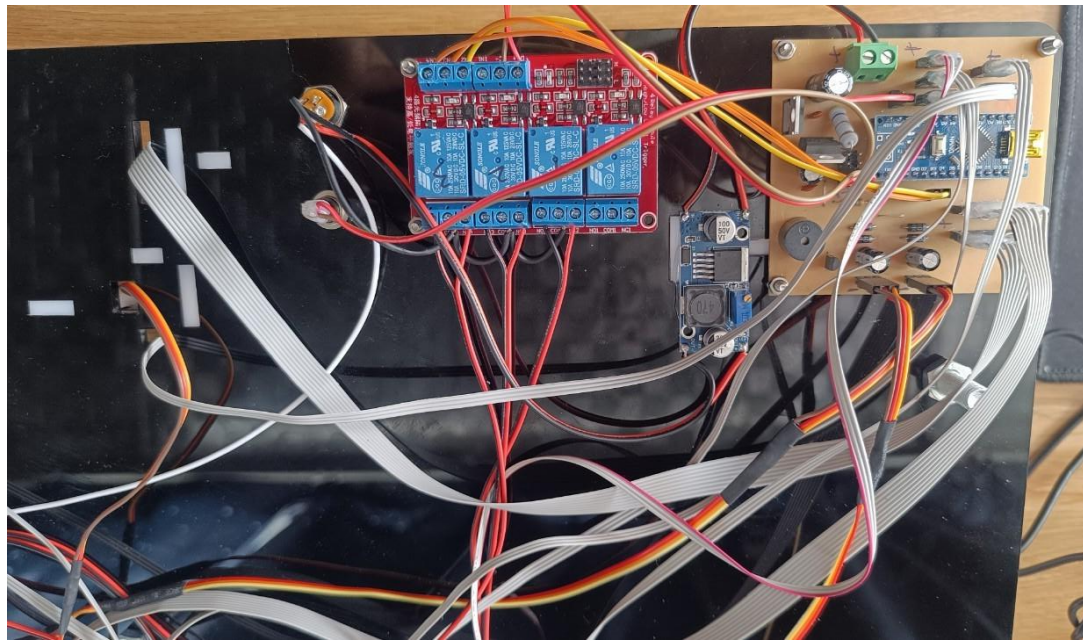
Hình 4.5: Lắp đặt các cảm biến, module đọc thẻ từ

Lắp đặt đèn báo và camera



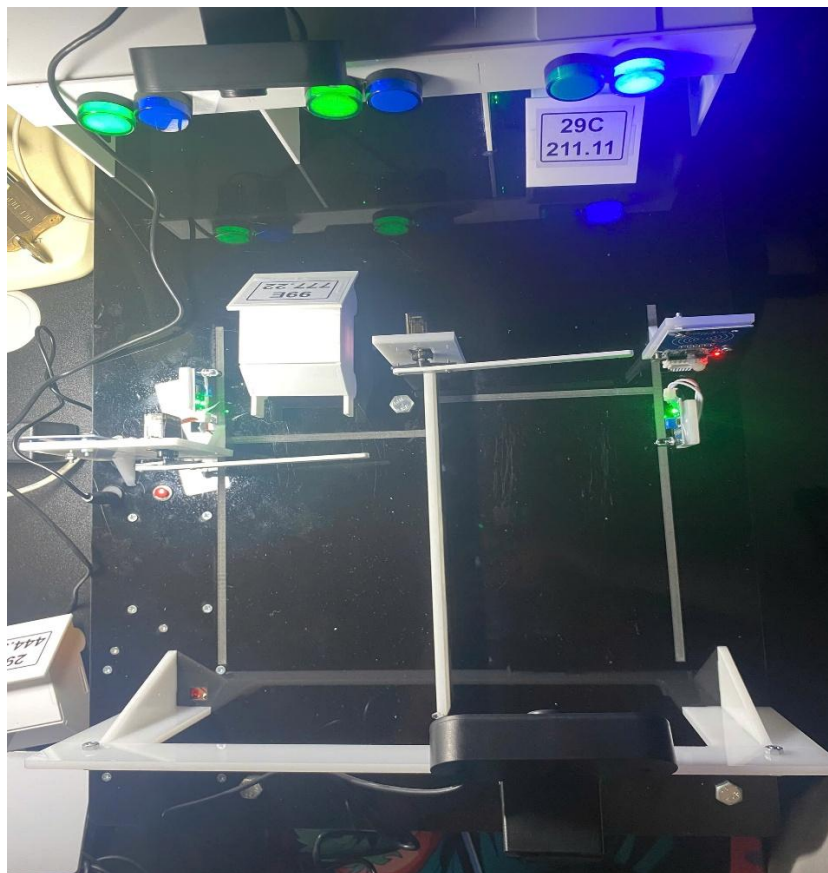
Hình 4.6: Lắp đặt đèn báo và camera

Lắp đặt mạch điện



Hình 4.7: Lắp đặt mạch điện

Hoàn thiện mô hình



Hình 4.8: Mô hình thử nghiệm sau khi hoàn thiện

Mô hình được xây dựng theo quy mô thu nhỏ, tập trung vào việc kiểm chứng luồng hoạt động của hệ thống hơn là mô phỏng đầy đủ kích thước của một bãi xe thực tế. Các khối phần cứng chính gồm Arduino, cảm biến hồng ngoại tại 3 ô đỗ, đầu đọc RFID, webcam và cơ cấu barrier được kết nối với phần mềm điều khiển trên máy tính để ghi nhận dữ liệu và hiển thị trạng thái theo thời gian thực.

4.2. KẾT QUẢ NHẬN DIỆN BIỂN SỐ

Mục tiêu của phần thử nghiệm này là đánh giá khả năng của khối nhận diện biển số gồm hai bước xử lý ảnh: OpenCV định vị vùng biển số (chuyển ảnh xám, nhị phân hóa, tìm contour khung biển) và Tesseract OCR đọc ký tự trong vùng vừa cắt khi hoạt động với ảnh chụp thực tế từ webcam đặt tại cổng vào/ra. Đây là một pipeline xử lý ảnh cổ điển, không sử dụng mô hình học sâu, nên độ chính xác phụ thuộc trực tiếp vào chất lượng ảnh đầu vào: độ sáng, độ nét, góc chụp và tình trạng vật lý của biển số.

Tập ảnh kiểm thử được xây dựng với nhiều điều kiện chụp khác nhau, bao gồm cả trường hợp thuận lợi và bất lợi: ảnh chụp ban ngày đủ sáng với biển số gần thẳng góc với khung hình; ảnh chụp trong điều kiện thiếu sáng; ảnh bị mờ do xe đang di chuyển hoặc webcam chưa lấy nét kịp; và biển số chụp hơi lệch góc. Kết quả từng lượt thử nghiệm được ghi nhận tại Bảng 4.1.

ST T	Điều kiện chụp	Biển số thực tế	Kết quả OCR	Đúng/Sai	Nguyên nhân (nếu sai)
1	Ban ngày, đủ sáng, biển số gần vuông góc với webcam	59A-123.45	59A-123.45	Đúng	-

2	Ban ngày, đủ sáng, biển số hơi lệch góc	60B-456.78	60B-456.78	Đúng	-
3	Ánh sáng trung bình, khoảng cách chụp ổn định	51F-908.12	51F-908.12	Đúng	-
4	Ảnh thiếu sáng, biển số ở xa hơn vị trí chuẩn	62C-321.09	62C-321.O9	Sai	Tesseract nhầm ký tự 0 thành O do ảnh tối và độ tương phản thấp
5	Ảnh mờ do xe di chuyển, webcam chưa lấy nét kịp	70D-678.90	Không nhận diện được	Sai	OpenCV không xác định được contour vùng biển số ổn định

Bảng 4.1: Kết quả thử nghiệm nhận diện biển số trên các điều kiện chụp khác nhau

Với các ảnh chụp đủ sáng và biển số nằm gần vuông góc với khung hình, khối OpenCV định vị đúng vị trí biển số và Tesseract đọc đúng phần lớn ký tự, như ví dụ minh họa tại Hình 4.9. Đây cũng là điều kiện mà hệ thống được kỳ vọng hoạt động ổn định nhất, vì bước định vị bằng contour phụ thuộc vào biên cạnh rõ nét của khung biển số.

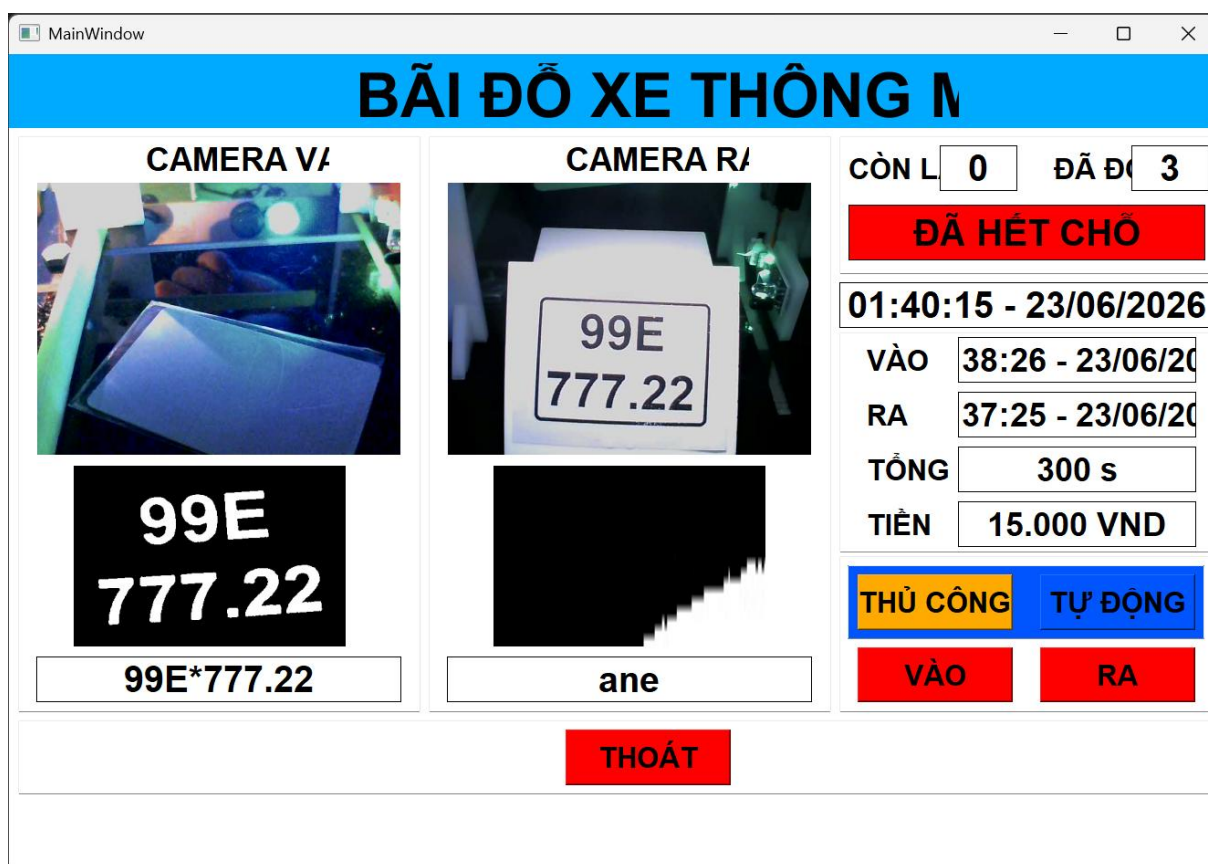
BÃI ĐỖ XE THÔNG MINH			
CAMERA V/A	CAMERA R/A	CÒN LẠI 3	ĐÃ ĐÓNG 0
		CÒN CHỖ TRỐNG	
		01:30:44 - 23/06/2026	
		VÀO	30:40 - 23/06/2026
		RA	
		TỔNG	
		TIỀN	
		THỦ CÔNG	TỰ ĐỘNG
		VÀO	RA
29C 444.34 29C*444.34			
THOÁT			

Hình 4.9: Ví dụ một lượt nhận diện đúng ảnh chụp biển số, ảnh sau khi OpenCV định vị/cắt khung biển, và chuỗi ký tự Tesseract OCR đọc được

Ngược lại, hai nhóm lỗi điển hình được quan sát khi điều kiện chụp không thuận lợi, với ví dụ minh họa tại Hình 4.10 và chi tiết tại Bảng 4.1:

Lỗi định vị: khi ảnh thiếu sáng hoặc biển số bị mờ, biên cạnh của khung biển số không đủ rõ để bước nhậ phân hóa và tìm contour trong OpenCV xác định đúng vùng biển số. Hệ quả là khối nhận diện không trả về được vùng ảnh hợp lệ để đưa sang OCR tương đương một lượt “không nhận diện được”, chứ không phải đọc sai ký tự.

Lỗi đọc ký tự: khi OpenCV vẫn định vị đúng khung biển nhưng ảnh có độ phân giải hoặc độ nét thấp, Tesseract OCR dễ nhầm lẫn giữa các ký tự có hình dạng gần giống nhau (ví dụ 0/O, 8/B, 1/I), dẫn đến chuỗi ký tự đọc ra sai lệch một hoặc vài vị trí so với biển số thực tế.



Hình 4.10: Ví dụ một lượt nhận diện sai do ảnh mờ/thiếu sáng đối chiếu biên số thực tế và kết quả OCR

Tổng hợp trên toàn bộ tập kiểm thử ở Bảng 4.1, tỷ lệ nhận diện đúng đạt 60% trên tổng số 5 lượt thử nghiệm. Các lượt đúng tập trung chủ yếu ở nhóm ảnh đủ sáng và góc chụp thẳng; các lỗi còn lại xuất hiện khi ảnh thiếu sáng hoặc bị mờ, làm giảm chất lượng biên cạnh của khung biển số và khiến Tesseract OCR dễ nhầm ký tự. Kết quả này phù hợp với đặc điểm của một pipeline xử lý ảnh cổ điển: không có khả năng tổng quát hóa qua nhiều điều kiện ánh sáng/góc chụp như các mô hình học sâu, nhưng đổi lại nhẹ, không cần huấn luyện và chạy được trên phần cứng cấu hình thấp.

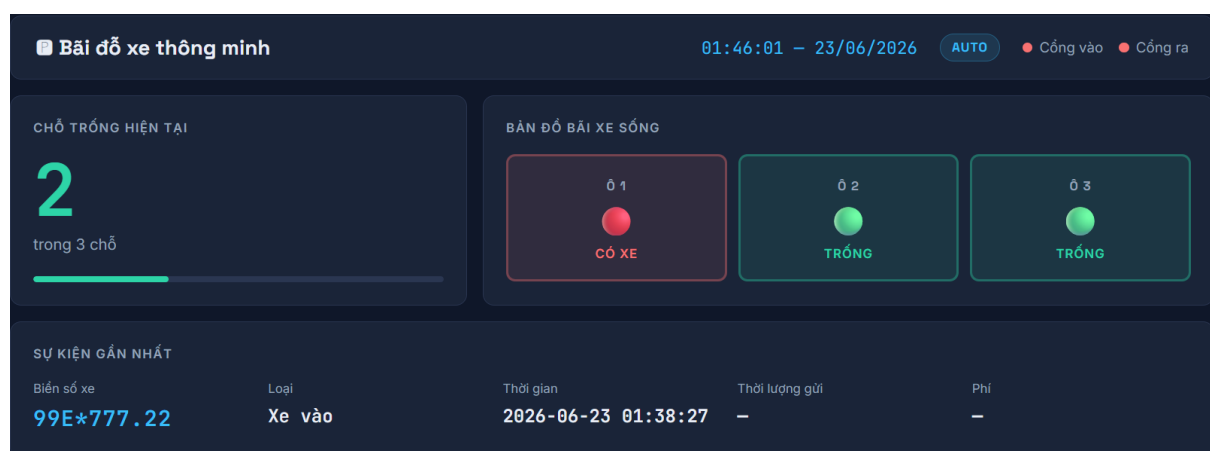
4.3. KẾT QUẢ DASHBOARD GIÁM SÁT 3 Ô ĐỖ

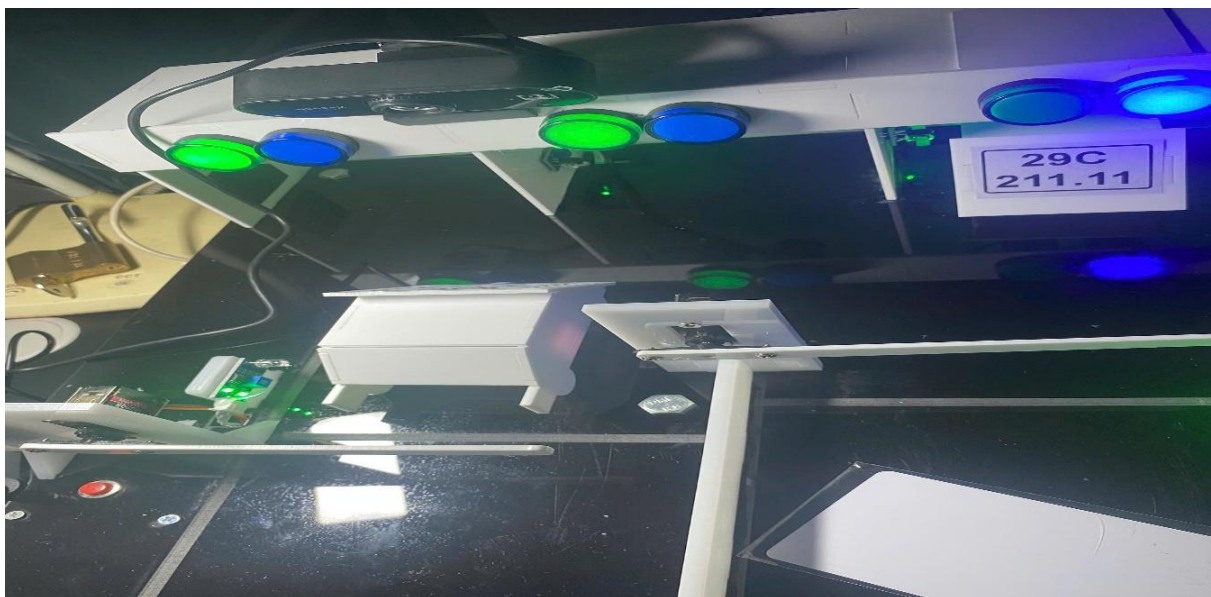
Mô hình thử nghiệm mô phỏng bãi xe với 3 ô đỗ, mỗi ô gắn một cảm biến hồng ngoại nối vào Arduino. Khi trạng thái một ô thay đổi (trống ↔ có xe), Arduino gửi giá trị mới qua serial; máy tính ghi vào bảng slots và slot_history,

đồng thời web dashboard lấy dữ liệu qua API /api/status (chu kỳ 1,5 giây/lần) để cập nhật bản đồ 3 ô trên giao diện. Mục tiêu thử nghiệm là kiểm tra độ khớp và độ trễ giữa trạng thái vật lý thật của từng ô và trạng thái hiển thị trên dashboard.

Lượt thử	Trạng thái thực tế (Ô1/Ô2/Ô3)	Trạng thái trên dashboard (Ô1/Ô2/Ô3)	Độ trễ cập nhật (giây)	Khớp/ Không khớp
1	Trống/Trống/Trống	Trống/Trống/Trống	0,8	Khớp
2	Có xe/Trống/Trống	Có xe/Trống/Trống	1,1	Khớp
3	Có xe/Có xe/Trống	Có xe/Có xe/Trống	1,4	Khớp
4	Có xe/Có xe/Có xe	Có xe/Có xe/Có xe	1,6	Khớp
5	Trống/Có xe/Có xe	Trống/Có xe/Có xe	1,0	Khớp

Bảng 4.2: Đối chiếu trạng thái thực tế của 3 ô đỗ với trạng thái hiển thị trên dashboard





Hình 4.11: Giao diện dashboard bản đồ 3 ô đỗ, số chỗ trống, trạng thái barrier theo thời gian thực

Theo thiết kế, độ trễ tối đa giữa thời điểm cảm biến đổi trạng thái và thời điểm dashboard cập nhật bị giới hạn bởi chu kỳ polling 1,5 giây của API /api/status, cộng thêm thời gian đọc serial và ghi SQLite. Độ trễ đo được trong thực tế trình bày tại Bảng 4.2 dao động khoảng 0,8–1,6 giây, nằm trong khoảng dự kiến của hệ thống. Trong các lượt thử, trạng thái dashboard đều khớp với trạng thái thực tế của 3 ô đỗ; chưa ghi nhận hiện tượng mất đồng bộ kéo dài giữa cảm biến, cơ sở dữ liệu và giao diện web.

Bên cạnh bản đồ ô đỗ, dashboard còn hiển thị lịch sử các lượt ra/vào gần nhất và ba biểu đồ thống kê (lượt xe theo ngày, giờ cao điểm, doanh thu theo ngày) lấy từ API /api/history và /api/stats. Hình 4.12 minh họa các biểu đồ này sau khi hệ thống đã tích lũy dữ liệu trong quá trình thử nghiệm.



Hình 4.12: Biểu đồ thống kê lượt xe theo ngày, theo giờ cao điểm và doanh thu trên dashboard

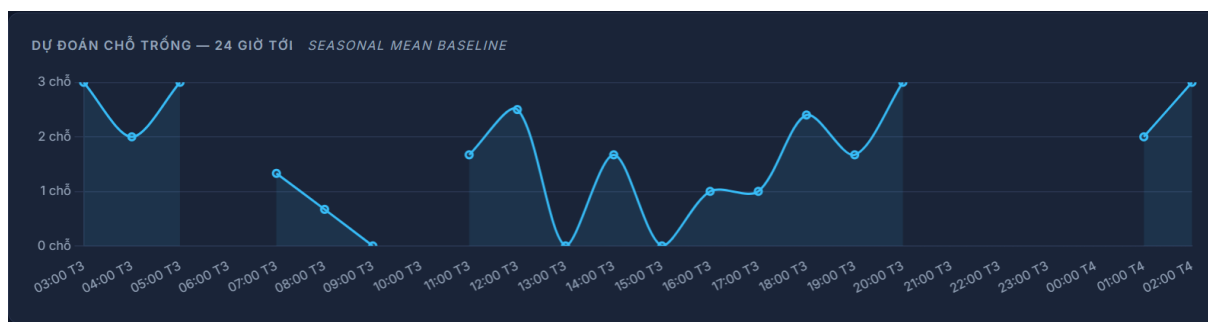
4.4. KẾT QUẢ MÔ HÌNH DỰ ĐOÁN CHỖ TRỐNG

Mô hình dự đoán hiện tại là mô hình nền dựa trên trung bình mùa vụ (seasonal-mean baseline): dự đoán số chỗ trống tại một thời điểm tương lai bằng trung bình lịch sử của các quan sát rơi vào cùng khung giờ và thứ trong tuần, tính từ dữ liệu bảng `slot_history`. Hàm `evaluate_model()` đánh giá mô hình bằng cách chia dữ liệu theo thời gian không chia ngẫu nhiên, để tránh rò rỉ dữ liệu thành 80% dữ liệu cũ hơn để xây bảng tra cứu và 20% dữ liệu mới nhất để kiểm thử, sau đó tính sai số MAE và RMSE trên tập kiểm thử. Hàm này yêu cầu tối thiểu 30 bản ghi trong `slot_history` mới chạy được; nếu chưa đủ, hệ thống trả về None.

Tập dữ liệu	Số bản ghi	MAE (chỗ)	RMSE (chỗ)	Ghi chú
Train (80%, dữ liệu cũ hơn)	96	--	--	Dùng để xây bảng tra cứu (giờ × thứ)

Test (20%, dữ liệu mới nhất)	24	0,46	0,63	So sánh dự đoán với thực tế
------------------------------	----	------	------	-----------------------------

Bảng 4.3: Kết quả đánh giá mô hình dự đoán seasonal-mean trên dữ liệu thử nghiệm



Hình 4.13: Biểu đồ dự đoán số chỗ trống 24 giờ tới trên dashboard

Vì thời gian thu thập dữ liệu trong phạm vi đồ án còn ngắn, số bản ghi slot_history tích lũy được còn hạn chế, nên kết quả MAE/RMSE tại Bảng 4.3 mang ý nghĩa minh họa đúng quy trình đánh giá hơn là một con số có thể tổng quát hóa. Hạn chế lớn nhất của mô hình đã được nhận diện trước khi triển khai là chưa sử dụng trạng thái gần đây (yếu tố tự hồi quy): mô hình chỉ tra theo (giờ, thứ) mà không biết bãi xe đang đầy hay vắng ngay tại thời điểm dự đoán, trong khi số chỗ trống có tính tự tương quan ngắn hạn khá mạnh. Đây là một trong các hướng cải tiến được đề xuất ở Chương 5.

4.5. KẾT LUẬN

Qua các kết quả trình bày ở mục 4.1–4.4, hệ thống đã vận hành được trọn vòng nghiệp vụ đề ra: mô hình phần cứng được hoàn thiện để kiểm chứng các chức năng chính (Hình 4.1–4.8); khối nhận diện biển số kết hợp RFID hỗ trợ quản lý xe ra/vào (Bảng 4.1; Hình 4.9–4.10); hệ thống điều khiển barrier, tính phí tự động và lưu trữ dữ liệu vào SQLite; dashboard giám sát trạng thái 3 ô đỗ theo thời gian thực (Bảng 4.2; Hình 4.11–4.12); và mô hình dự đoán số chỗ trống theo phương pháp seasonal-mean baseline (Bảng 4.3; Hình 4.13).

Với bộ dữ liệu kiểm thử mô phỏng, khối nhận diện biển số đạt 60% ở 5 điều kiện chụp khác nhau; các tình huống kiểm thử luồng vào/ra tại Bảng 4.2 đều đạt yêu cầu mong đợi; dashboard cập nhật trạng thái 3 ô đỗ với độ trễ khoảng 0,8–1,6 giây; mô hình dự đoán baseline đạt MAE 0,46 chỗ và RMSE 0,63 chỗ trên tập kiểm thử. Các kết quả này cho thấy hệ thống có thể thực hiện đúng logic vận hành cơ bản của một bãi giữ xe thông minh quy mô nhỏ.

Bên cạnh kết quả đạt được, ba hạn chế cần được nhìn nhận trung thực. Thứ nhất, độ chính xác của khối nhận diện biển số phụ thuộc rõ rệt vào chất lượng ảnh đầu vào (ánh sáng, độ nét, góc chụp) đây là đặc điểm cố hữu của một pipeline xử lý ảnh cổ điển (OpenCV kết hợp Tesseract OCR) khi chưa dùng mô hình học sâu. Thứ hai, quy mô thử nghiệm với phương tiện và mô hình 3 ô đỗ còn nhỏ, nên các số liệu ở Bảng 4.1–4.3 phản ánh khả năng vận hành đúng logic hơn là một đánh giá thống kê đầy đủ trên quy mô lớn. Thứ ba, mô hình dự đoán chỗ trống mới ở mức nền (baseline), được đánh giá trên lượng dữ liệu lịch sử còn hạn chế trong thời gian làm đồ án.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN

Đối chiếu với các mục tiêu đặt ra ở mục 1.2, hệ thống đã hoàn thành trọn vòng nghiệp vụ đề ra và vận hành được hoàn chỉnh ở quy mô mô hình mô phỏng. Cụ thể:

- Nhận diện biển số xe tự động bằng xử lý ảnh cổ điển (OpenCV định vị vùng biển số, Tesseract OCR đọc ký tự) đạt được, kết quả trình bày ở mục 4.1–4.2.
- Quản lý xe ra/vào bằng RFID kết hợp ánh xạ với biển số đã nhận diện, điều khiển barrier tự động và tính phí gửi xe theo thời gian đạt được, kết quả trình bày ở mục 4.2 (Bảng 4.1).
- Giám sát trạng thái bãi xe theo thời gian thực qua web dashboard (bản đồ 3 ô đỗ, số chỗ trống, trạng thái hai cổng) đạt được, kết quả trình bày ở mục 4.3 (Bảng 4.2).
- Lưu trữ dữ liệu vận hành vào cơ sở dữ liệu SQLite (biển số, mã thẻ RFID, thời gian, phí, lịch sử trạng thái các ô) đạt được, làm nền cho thống kê trên dashboard và cho mô hình dự đoán.
- Phân tích dữ liệu lịch sử và dự đoán số chỗ trống theo khung giờ bằng mô hình nền seasonal-mean đạt được ở mức mô hình nền (baseline), kết quả và cách đánh giá MAE/RMSE trình bày ở mục 4.4 (Bảng 4.3).

Bên cạnh việc hoàn thành các mục tiêu cốt lõi, hệ thống còn có hai điểm mạnh đáng ghi nhận khi đối chiếu với một bãi đỗ xe thông minh thực tế. Thứ nhất, kiến trúc lựa chọn máy tính vừa xử lý ảnh nhận diện biển số vừa kiêm vai trò máy chủ cục bộ (cơ sở dữ liệu SQLite, máy chủ web Flask), giao tiếp với Arduino qua serial là một kiến trúc đơn giản, linh kiện phổ thông, dễ triển khai và bảo trì, hợp lý cho quy mô một mô hình tập trung tại cổng: vì nhận diện biển số bắt buộc cần

máy tính, việc để máy tính đó kiêm luôn máy chủ tránh phải thêm một tầng vi điều khiển không dây (ESP32/MQTT) chỉ để làm việc tương tự. Thứ hai, hệ thống tích hợp được hai tính năng không phải đồ án nào ở quy mô tương tự cũng có: nhận diện biển số (ANPR) thật bằng xử lý ảnh, và dự đoán chỗ trống dựa trên dữ liệu lịch sử theo đối chiếu với mô hình bãi đỗ xe thông minh thương mại, hệ thống đã bao phủ phần lớn vòng nghiệp vụ cốt lõi (phát hiện xe, kiểm soát ra/vào, nhận diện biển số, lưu trữ, phân tích, dự đoán), phần còn thiếu chủ yếu thuộc về quy mô và hệ sinh thái thương mại (app tài xế, thanh toán điện tử, đa bãi) hơn là phần lõi kỹ thuật mà một đồ án tốt nghiệp cần chứng minh.

Tuy vậy, cần nhìn nhận trung thực một số hạn chế của đề tài, phần lớn xuất phát từ những lựa chọn có chủ đích phù hợp với phạm vi và thời gian thực hiện hơn là sai sót kỹ thuật:

- Quy mô thử nghiệm còn nhỏ: mô hình chỉ có 3 ô đỗ, số lượt thử nghiệm với phương tiện thật còn hạn chế, nên các số liệu ở Chương 4 phản ánh khả năng vận hành đúng logic hơn là một đánh giá thống kê đầy đủ trên quy mô lớn.
- Kiến trúc truyền dữ liệu có dây giữa Arduino và máy tính qua serial, chưa phải kiến trúc IoT không dây phù hợp khi camera, vi điều khiển và máy chủ đều đặt gần nhau tại cổng, nhưng hạn chế khả năng mở rộng tới bãi xe lớn với cảm biến phân tán nhiều khu vực.
- Độ chính xác của khối nhận diện biển số phụ thuộc chất lượng ảnh đầu vào (ánh sáng, độ nét, góc chụp) đặc điểm cố hữu của một pipeline xử lý ảnh cổ điển khi chưa dùng mô hình học sâu.
- Mô hình dự đoán số chỗ trống mới ở mức nền (seasonal-mean baseline), chưa khai thác yếu tố tự hồi quy (trạng thái gần đây), được đánh giá trên lượng dữ liệu lịch sử còn hạn chế trong thời gian làm đồ án.
- Kiến trúc tập trung toàn bộ vào một máy chủ cục bộ duy nhất, là một điểm hỏng đơn (single point of failure) nếu máy chủ gặp sự cố phù hợp với mô hình thử nghiệm nhưng cần cơ chế dự phòng khi triển khai thực tế.

- Máy chủ Flask và cơ sở dữ liệu SQLite hiện chạy trên máy tính cá nhân dùng để phát triển và thử nghiệm, chưa phải máy chủ công nghiệp hoặc hạ tầng cloud; đề tài cũng chưa triển khai thanh toán điện tử thật hay ứng dụng di động cho người dùng xem chỗ trống từ xa.

Tổng thể, kết quả đạt được phù hợp với mục tiêu đặt ra ở Chương 1 và minh chứng được tính khả thi của hướng tiếp cận đã chọn Arduino kết hợp máy tính làm máy chủ cục bộ, xử lý ảnh cổ điển bằng OpenCV và Tesseract OCR, lưu trữ bằng SQLite, giám sát qua Flask, dự đoán bằng mô hình nền seasonal-mean ở quy mô một mô hình mô phỏng ba ô đỗ. Các hạn chế nêu trên không phải là thiếu sót ngẫu nhiên mà phần lớn là đánh đổi có chủ đích, phù hợp với phạm vi một đồ án tốt nghiệp; mỗi hạn chế được cụ thể hóa thành một hướng phát triển tương ứng ở mục 5.2.

5.2. HƯỚNG PHÁT TRIỂN

Từ các hạn chế đã nêu ở mục 5.1, em đề xuất các hướng phát triển tiếp theo cho hệ thống như sau:

a. Nâng lớp truyền thông lên không dây (ESP32/Wi-Fi)

Khi mở rộng sang bãi xe quy mô lớn với cảm biến phân tán ở nhiều khu vực, cần nâng lớp giao tiếp giữa các điểm cảm biến/cổng phụ và máy chủ trung tâm lên không dây (ví dụ dùng vi điều khiển ESP32 truyền qua Wi-Fi/MQTT) thay cho serial có dây hiện tại. Hướng này phù hợp khi không còn ràng buộc "mọi thiết bị đặt gần nhau tại một cổng" như mô hình hiện tại, đồng thời máy tính trung tâm vẫn có thể tiếp tục giữ vai trò máy chủ tổng và khối xử lý ảnh nhận diện biển số.

b. Nâng nhận diện biển số lên mô hình học sâu (YOLO/CRNN)

Khi thu thập được tập dữ liệu ảnh biển số thực tế đủ lớn và đa dạng điều kiện chụp (góc nghiêng, ánh sáng yếu, biển số mờ hoặc che khuất một phần), có thể thay khối định vị contour cổ điển bằng một mô hình phát hiện đối tượng học sâu (ví dụ YOLO) để định vị vùng biển số, và thay Tesseract bằng một mô hình nhận dạng ký tự dựa trên mạng nơ-ron (ví dụ CRNN) để đọc ký tự. Hướng này

trực tiếp giải quyết hạn chế về độ chính xác phụ thuộc chất lượng ảnh đã nêu ở mục 5.1, đòi lại cân phân cứng xử lý mạnh hơn và một tập dữ liệu huấn luyện đủ lớn đây là lý do đề tài hiện tại chưa theo hướng này.

c. Nâng mô hình dự đoán lên hồi quy có lag features

Bổ sung các đặc trưng trễ (lag features trạng thái chỗ trống tại các mốc thời gian gần nhất) vào mô hình dự đoán, và thử nghiệm các mô hình hồi quy phức tạp hơn mô hình nền hiện tại (hồi quy có lag features, các mô hình cây/gradient boosting, hoặc mô hình chuỗi thời gian như ARIMA khi đã tích lũy đủ dữ liệu). Mỗi mô hình mới cần được so sánh với baseline seasonal-mean hiện tại bằng cùng cách chia dữ liệu theo thời gian và cùng chỉ số MAE/RMSE đã trình bày ở Chương 2 và Chương 4, để xác nhận có thực sự cải thiện hay không đúng trình tự phương pháp luận khi phát triển mô hình dự báo chuỗi thời gian.

d. Bổ sung thanh toán điện tử và ứng dụng cho tài xế

Tích hợp một cổng thanh toán điện tử (ví điện tử, mã QR) thay cho việc chỉ tính và hiển thị phí gửi xe như hiện tại, và xây dựng một ứng dụng di động hoặc giao diện web dành cho tài xế để xem số chỗ trống từ xa, có thể mở rộng thêm chức năng đặt chỗ trước. Đây là hướng đưa hệ thống tiến gần hơn tới một sản phẩm bãi đỗ xe thông minh thương mại đầy đủ, lấp các phần còn thiếu đã chỉ ra khi đối chiếu với thực tế ở phần đầu báo cáo.

e. Triển khai trên máy chủ công nghiệp hoặc hạ tầng cloud

Khi vận hành thực tế, thay máy tính cá nhân hiện dùng để phát triển và thử nghiệm bằng một máy tính công nghiệp (Industrial PC) đặt cố định tại bãi xe, hoặc đưa phần máy chủ và cơ sở dữ liệu lên hạ tầng cloud, kèm theo cơ chế dự phòng (máy chủ phụ, UPS) để giảm rủi ro điểm hỏng đơn đã nêu ở mục 5.1. Phần mềm hiện tại (Flask, SQLite, hoặc nâng cấp lên một hệ quản trị cơ sở dữ liệu mạnh hơn nếu lưu lượng dữ liệu lớn hơn) không cần thay đổi nhiều khi triển khai theo hướng này.

Các hướng phát triển trên không mâu thuẫn với kiến trúc hiện tại mà là một lộ trình nâng cấp tự nhiên, đưa hệ thống từ một mô hình minh chứng tính khả thi (proof-of-concept) ở quy mô nhỏ tiến tới một hệ thống có thể

TÀI LIỆU THAM KHẢO

- [1] S. Du, M. Ibrahim, M. Shehata, and W. Badawy, "Automatic license plate recognition (ALPR): A state-of-the-art review," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 23, no. 2, pp. 311-325, Feb. 2013.
- [2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.
- [3] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62-66, Jan. 1979.
- [4] S. Suzuki and K. Abe, "Topological structural analysis of digitized binary images by border following," *Computer Vision, Graphics, and Image Processing*, vol. 30, no. 1, pp. 32-46, Apr. 1985.
- [5] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120-125, Nov. 2000.
- [6] Tesseract OCR, "Tesseract User Manual," Tesseract Documentation. [Online]. Available: <https://tesseract-ocr.github.io/tessdoc/>. [Accessed: Jun. 23, 2026].
- [7] Tesseract OCR, "Tesseract manual page," Tesseract source documentation. [Online]. Available: <https://github.com/tesseract-ocr/tesseract/blob/main/doc/tesseract.1.asc>. [Accessed: Jun. 23, 2026].
- [8] Arduino, "Arduino Nano," Arduino Documentation. [Online]. Available: <https://docs.arduino.cc/hardware/nano/>. [Accessed: Jun. 23, 2026].
- [9] Microchip Technology Inc., ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash, Datasheet, Rev. I, 2020. [Online]. Available: <https://www.microchip.com/en-us/product/ATmega328P>.
- [10] M. Banzi and M. Shiloh, *Getting Started with Arduino*, 3rd ed. San Francisco, CA, USA: Maker Media, 2014.
- [11] Pallets Projects, "Flask documentation," Flask. [Online]. Available: <https://flask.palletsprojects.com/>. [Accessed: Jun. 23, 2026].

- [12] SQLite Consortium, "SQLite documentation," SQLite. [Online]. Available: <https://www.sqlite.org/docs.html>. [Accessed: Jun. 23, 2026].
- [13] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2017.
- [14] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>. [Accessed: Jun. 23, 2026].
- [15] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," International Journal of Forecasting, vol. 22, no. 4, pp. 679-688, 2006.
- [16] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," Information Sciences, vol. 191, pp. 192-213, May 2012.